

Technical University of Crete

School of Electrical & Computer Engineering

Diploma Thesis

Investigating the applicability of Federated Dynamic Averaging in Federated Low-Rank Adaptation Architectures.

Author:

Asterinos Karalis

Thesis Committee:

Assoc. Prof. Vasilis Samoladas (Supervisor)

Prof. Antonios Deligiannakis

Prof. Minos Garofalakis

A thesis submitted in partial fulfillment of the requirements
for the degree of Diploma in Electrical and Computer Engineering

October, 2026

Abstract

Federated Learning has established itself as a foundational paradigm for privacy-preserving collaborative model training across decentralized data sources. However, scaling modern foundation models to billions of parameters exposes a critical dual bottleneck in federated optimization: an excessive *spatial* communication overhead from transmitting high-dimensional parameter updates, and an inefficient *temporal* communication pattern imposed by rigid, fixed-interval synchronization schedules. Parameter-efficient fine-tuning methods such as Low-Rank Adaptation address the spatial dimension by decomposing updates into compact low-rank factors. Yet, naively applying Low-Rank Adaptation in federated environments introduces a fundamental structural inconsistency: independent averaging of constituent low-rank matrices fails to reconstruct the true global update due to the non-commutativity of matrix multiplication. Furthermore, existing federated Low-Rank Adaptation variants remain temporally inefficient by relying on static synchronization frequencies agnostic to local training dynamics.

To address both dimensions simultaneously, this thesis investigates combining Federated Dynamic Averaging with federated low-rank adaptation. Specifically, we develop and formulate FDA-RoLoRA, an algorithmic formulation that pairs dynamic, variance-driven averaging with Robust Low-Rank Adaptation. This method eliminates the bilinear aggregation inconsistency through alternating updates between down-projection and up-projection matrices, while dynamically regulating synchronization frequency via decoupled, dynamically updated per-matrix variance thresholds. We evaluate the proposed framework on the General Language Understanding Evaluation (GLUE) benchmark using the roberta-base model.

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, Assoc. Prof. Vasilis Samoladas, for his continuous guidance, insightful advice, and constructive feedback throughout this thesis. His courses in Distributed Systems and Machine Learning Systems were pivotal during my studies, providing the core foundation for this research.

I extend my sincere appreciation to the examination committee members, Prof. Antonios Deligiannakis and Prof. Minos Garofalakis, for their time, thorough review, and valuable observations on this work.

I am equally grateful to my friends for their support, encouragement, and the unforgettable moments shared throughout our university years in Chania.

Finally, I dedicate this thesis to my family. Their unconditional love, patience, and continuous support have been my strongest anchor throughout every step of this journey.

Contents

Abstract i

Acknowledgements iii

List of Figures vii

List of Tables ix

List of Algorithms xi

List of Abbreviations xiii

1 Introduction 1
- 1.1 Federated Learning and Optimization . . . 1
- 1.2 Foundation Models and Parameter-Efficient Fine-Tuning . . . 2
- 1.3 Aggregation Inconsistency in Federated LoRA . . . 3
- 1.4 Robust Low-Rank Adaptation . . . 3
- 1.5 Proposed Method: FDA-RoLoRA . . . 4
- 1.6 Thesis Outline . . . 5

2 Background 7
- 2.1 Neural Networks and Optimization . . . 7
  - 2.1.1 Neural Networks . . . 7
  - 2.1.2 Gradient Descent . . . 8
  - 2.1.3 Stochastic Gradient Descent . . . 8
- 2.2 Foundation Models and the Transformer Architecture . . . 8
- 2.3 Distributed Machine Learning . . . 9
- 2.4 Federated Learning and Communication Bottlenecks . . . 11
  - 2.4.1 Federated Averaging . . . 11
  - 2.4.2 FedOpt: Server-Side Adaptive Optimization . . . 12
  - 2.4.3 Client Drift and Statistical Heterogeneity . . . 13
  - 2.4.4 Dirichlet-Based Data Partitioning . . . 13
  - 2.4.5 Communication Cost Quantification . . . 13
- 2.5 Parameter-Efficient Fine-Tuning . . . 14
  - 2.5.1 The PEFT Landscape . . . 14
  - 2.5.2 Low-Rank Adaptation (LoRA) . . . 15
  - 2.5.3 Structural and Dynamic Asymmetry in LoRA Matrices . . . 16
- 2.6 Federated Low-Rank Adaptation . . . 16
- 2.7 Robust Low-Rank Adaptation . . . 17

2.8 Federated Dynamic Averaging . . . 18
2.8.1 Geometric Monitoring Foundations . . . 18

3 Methodology **21**
3.1 Combining Dynamic Averaging with Federated LoRA . . . 21
3.2 Foundational Precursor Protocols: RoLoRA and FDA . . . 22
3.3 Notation and Hyperparameters . . . 24
3.4 FDA-RoLoRA Initialization and Subspace Toggling . . . 26
3.5 Local Training and Direct Subspace Variance . . . 26
3.6 Server-Coordinated Synchronization Halt and Calibration . . . 27
3.7 Global Aggregation and Independent Threshold Evolution . . . 27
3.7.1 Numerical Walkthrough of Dual-Track Evolution . . . 28
3.8 Server Optimization and Pseudo-Gradient Dynamics . . . 29

4 Implementation **31**
4.1 Software Architecture . . . 31
4.2 Data Partitioning . . . 32
4.3 Simulation Controls and Early Stopping . . . 32
4.4 Model Configuration . . . 33
4.5 Training Pipeline . . . 34
4.6 Deployment Environment . . . 35

5 Evaluation **37**
5.1 Experimental Setup for Foundation Models . . . 37
5.1.1 Benchmark Tasks and Corpus Standardization . . . 37
5.1.2 Statistical Data Partitioning . . . 38
5.1.3 Accuracy-Gated Early Stopping Protocol . . . 38
5.2 Comparative Baselines . . . 39
5.3 Foundation Model Evaluation and Empirical Benchmarks . . . 39
5.3.1 Homogeneous Benchmark Suite (IID Settings, $\alpha = 1000.0$) . . . 39
5.3.2 Statistical Heterogeneity Benchmark Suite (Non-IID Settings, Dirichlet Skew $\alpha = 0.5$) . . . 41
5.3.3 Cross-Setting Synthesis: Communication and Geometric Stability . . . 42

6 Conclusion and Future Work **45**
6.1 Synthesis of Results and Methodological Takeaways . . . 45
6.2 Limitations . . . 46
6.3 Future Work . . . 46

Bibliography **49**

List of Figures

2.1 Parameter Server Architecture for Asynchronous Distributed Learning. Workers independently pull the latest parameters and push computed gradients. 10
2.2 Chronological sequence of a Federated Learning communication round (broadcast, local client training, update collection, and server aggregation). Figure reproduced from Theologitis et al. [38]. 11
2.3 Low-Rank Adaptation (LoRA) schematic: decomposition of weight updates into low-rank matrices A and B alongside frozen pretrained weights W. . . 15
2.4 Robust Low-Rank Adaptation (RoLoRA) alternating optimization scheme. In each round, one low-rank matrix is frozen globally while the other is trained and aggregated linearly. 17

List of Tables

3.1 Summary of mathematical notation and system hyperparameters. 24

5.1 Comparative performance metrics on foundation model benchmarks (`roberta-base` on GLUE tasks, $C = 20$ clients) across evaluated baselines and the proposed FDA-RoLoRA framework. 39
5.2 Empirical evaluation on the homogeneous benchmark suite (IID, Dirichlet $\alpha = 1000.0$) across GLUE sequence classification tasks on `roberta-base`. . 40
5.3 Empirical evaluation on the statistical heterogeneity benchmark suite (Non-IID Dirichlet $\alpha = 0.5$) across GLUE tasks on `roberta-base`. 41

List of Algorithms

1 FDA 23
2 RoLoRA 23
3 FDA-RoLoRA 25

List of Abbreviations

Abbreviation	Description
Adam	Adaptive Moment Estimation
AdamW	Adam with Decoupled Weight Decay
AMS	Alon-Matias-Szegedy
FDA	Federated Dynamic Averaging
FDA-Opt	Federated Dynamic Averaging Optimizer
FDA-RoLoRA	Federated Dynamic Averaging with Robust Low-Rank Adaptation
FedAvg	Federated Averaging
FedLoRA	Federated Low-Rank Adaptation
FedOpt	Federated Optimization
FFA-LoRA	Federated Freeze-A Low-Rank Adaptation
FL	Federated Learning
FLoRA	Heterogeneous Federated Low-Rank Adaptation
GLUE	General Language Understanding Evaluation
IID	Independent and Identically Distributed
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MNLI	Multi-Genre Natural Language Inference
non-IID	Non-Independent and Identically Distributed
PEFT	Parameter-Efficient Fine-Tuning
RoLoRA	Robust Low-Rank Adaptation
SGD	Stochastic Gradient Descent
SLoRA	Sparse Low-Rank Adaptation
SST-2	Stanford Sentiment Treebank (Binary)

Chapter 1

Introduction

Federated Learning (FL) [17,28] has established itself as the leading paradigm for training machine learning models across decentralized data owners without requiring the centralization of raw user data. By enabling participating nodes—ranging from resource-constrained edge devices to institutional data silos such as hospitals and financial institutions—to compute updates locally and share only parameter tensors with a coordinating server, FL enables collaborative optimization while respecting data privacy regulations (e.g., GDPR, HIPAA) and proprietary commercial boundaries.

However, the iterative nature of federated optimization introduces a severe communication bottleneck. Historically, research in distributed machine learning has tackled this challenge primarily through temporal and algorithmic mechanisms designed for standard neural architectures. Yet, the recent paradigm shift toward massive foundation models has created an entirely new spatial crisis, rendering classical federated optimization techniques insufficient on their own.

1.1 Federated Learning and Optimization

In canonical federated optimization, the central communication bottleneck is dictated by the synchronization frequency between clients and the central server. The foundational Federated Averaging (FedAvg) algorithm [28] coordinates training by having clients execute a fixed, predetermined number of local stochastic gradient descent (SGD) epochs (E) before transmitting their updated parameter vectors to the server for aggregation.

While conceptually simple, rigid synchronization cadences suffer from fundamental inefficiencies under real-world conditions:

- **Statistical Heterogeneity and Client Drift:** In decentralized environments, local client datasets are inherently non-identically and independently distributed (non-IID) [18, 23]. When clients execute multiple uncoordinated local epochs, their local models drift toward local empirical minima that diverge sharply from the global optimization trajectory.

- **Temporal Redundancy:** Enforcing a static schedule remains blind to actual client convergence. In phases where local updates are minor or homogeneous, transmitting parameters after every fixed interval wastes critical network bandwidth on redundant updates. Conversely, when clients diverge rapidly, static schedules delay aggregation, exacerbating model divergence.

To improve federated optimization, adaptive server-side optimizers were introduced under the Federated Optimization (FedOpt) framework [30]. By interpreting aggregated client updates as pseudo-gradients and applying momentum-based tracking (such as FedAdam and FedYogi), FedOpt stabilizes the global optimization trajectory against client drift.

Subsequently, to resolve the rigid cadence of static rounds, Federated Dynamic Averaging (FDA-Opt) [11,38] introduced variance-driven dynamic synchronization. Rather than synchronizing after fixed epochs, FDA-Opt continuously monitors the pairwise drift and variance across participating clients. Synchronization is triggered dynamically only when parameter variance breaches an adaptive drift threshold (θ). By synchronizing strictly when mathematically necessary, FDA-Opt proved that dynamic temporal throttling can dramatically reduce total communication rounds without sacrificing final accuracy.

Crucially, however, FedAvg, FedOpt, and FDA-Opt all share a common architectural assumption: *the transmission and synchronization of full-rank model parameter vectors.*

1.2 Foundation Models and Parameter-Efficient Fine-Tuning

In recent years, the machine learning landscape has undergone an unprecedented transformation driven by large-scale pre-trained foundation models and Large Language Models (LLMs) scaling to billions of parameters [4, 6]. Adapting these foundation models to specialized domain tasks (e.g., clinical diagnosis, legal discovery, financial fraud detection) is of paramount importance.

Yet, this shift in model scale fundamentally breaks the physical viability of full-parameter federated learning:

- Synchronizing a modern 7-billion parameter language model (such as LLaMA-2-7B [39]) in 16-bit half-precision requires transmitting approximately **14 Gigabytes** of parameter data per client per round.
- In a cross-silo federated setting with 50 participating institutions over a modest horizon of 100 communication rounds, total network transfer exceeds **70 Terabytes**—an operational volume that is wholly intractable over standard wide-area network (WAN) connections.
- Even for moderately sized encoders such as RoBERTa-Large [26] (355M parameters), a single full-parameter round requires $\sim$1.4 GB per client, swamping edge up-links.

To overcome this spatial bottleneck, Parameter-Efficient Fine-Tuning (PEFT) has emerged as an indispensable methodology. Among PEFT strategies, Low-Rank Adaptation (LoRA) [16] has become the de facto standard. LoRA keeps the large pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ frozen, and factorizes task-specific parameter updates into two low-rank matrices:

$$W = W_0 + \Delta W = W_0 + AB \tag{1.1}$$

where $A \in \mathbb{R}^{d\times r}$ denotes the down-projection matrix, $B \in \mathbb{R}^{r\times k}$ denotes the up-projection matrix, and the rank $r \ll \min(d, k)$ (typically $r \in \{4, 8, 16\}$). Because only A and B are updated during training, LoRA compresses the volume of communicated parameters by over 99%, providing a powerful spatial solution for edge and silo environments.

1.3 Aggregation Inconsistency in Federated LoRA

Given the spatial efficacy of LoRA, directly integrating low-rank adapters into federated learning—often termed Federated LoRA (FedLoRA) or FedIT [45, 47]—appears to be a natural path forward. In naive FedLoRA, each client $i \in \{1, \ldots, N\}$ initializes identical low-rank matrices, computes local gradient updates on private data, and transmits only its trained adapter matrices A_i and B_i to the central server.

However, aggregating independently trained low-rank factors via standard FedAvg introduces a fatal mathematical failure mode: *LoRA aggregation inconsistency* (or aggregation discordance) [45]. Because matrix multiplication is non-linear and non-commutative, averaging the factor matrices independently across clients fails to reconstruct the true average of their composite weight updates:

$$\frac{1}{N}\sum_{i=1}^{N}(A_i B_i) \neq \left(\frac{1}{N}\sum_{i=1}^{N} A_i\right)\left(\frac{1}{N}\sum_{i=1}^{N} B_i\right) \tag{1.2}$$

When the central server independently averages A_i and B_i, the composite product of the averaged matrices generates cross-term interference vectors that do not exist in any client's local optimization trajectory. This bilinear distortion collapses the low-rank representation, induces severe gradient cancellation, and destabilizes global convergence, frequently causing fine-tuning to perform worse than static heuristics or simple localized training.

1.4 Robust Low-Rank Adaptation

To resolve this bilinear aggregation discordance, several structural workarounds have been explored in recent literature:

- **Full Weight Reconstruction:** Clients could theoretically reconstruct the dense update $\Delta W_i = A_i B_i$ before transmission, or the server could perform Singular Value Decomposition (SVD) on the aggregated full matrix [45]. However, this completely nullifies the spatial communication savings of LoRA and imposes prohibitive computational overhead.

- **Static Down-Projection Freezing (FFA-LoRA):** FFA-LoRA [36] permanently freezes the down-projection matrix A as a random projection across all clients, allowing clients to train and transmit only the up-projection matrix B. While this linearizes aggregation ($\frac{1}{N}\sum AB_i = A\frac{1}{N}\sum B_i$), permanently freezing half of the parameter degrees of freedom severely compromises adapter capacity and expressive power.

- **Heterogeneous Rank Assembly (FLoRA):** FLoRA [43] attempts to support variable client ranks by stacking updates, but still incurs heavy reconstruction overhead on the aggregation server.

A decisive breakthrough resolving this dilemma without sacrificing model expressiveness is **Robust Low-Rank Adaptation (RoLoRA)** [7]. RoLoRA adopts an *alternating coordinate optimization* strategy. Instead of updating both factor matrices simultaneously, training alternates in structured multi-round blocks:

1. In Phase A, matrix A is frozen globally across all clients, and only matrix B is trained locally and aggregated.

2. In Phase B, the updated matrix B is frozen globally across all clients, and only matrix A is trained locally and aggregated.

Because one coordinate factor is held globally identical across all clients during any given aggregation phase, the averaging operation becomes strictly linear:

$$\frac{1}{N}\sum_{i=1}^{N}(A_{\text{frozen}}B_i) = A_{\text{frozen}}\left(\frac{1}{N}\sum_{i=1}^{N}B_i\right) \tag{1.3}$$

By toggling the active subspace over alternating cycles, RoLoRA preserves the full representational rank and expressive capacity of standard LoRA while restoring mathematically exact federated averaging.

1.5 Proposed Method: FDA-RoLoRA

With RoLoRA restoring mathematical linearity to federated low-rank aggregation, an extraordinary scientific opportunity emerges: *Can we now unify the temporal adaptivity of Federated Dynamic Averaging with the spatial efficiency of alternating low-rank adaptation?*

While RoLoRA successfully solves the spatial aggregation inconsistency, existing RoLoRA formulations remain shackled to **rigid, predetermined synchronization cadences** (fixed local steps or epochs). Clients must synchronize their active matrix after arbitrary epoch boundaries, inheriting all the temporal redundancies and drift penalties of classical FedAvg.

Conversely, while FDA-Opt [38] pioneered variance-driven dynamic synchronization, it was formulated exclusively for monolithic full-rank neural networks. Simply applying FDA-Opt directly to RoLoRA is impossible due to three fundamental obstacles:

1. **Asymmetric Coordinate Geometries:** The down-projection matrix $A \in \mathbb{R}^{d\times r}$ and up-projection matrix $B \in \mathbb{R}^{r\times k}$ fulfill entirely different functional roles (compression vs. expansion). Furthermore, standard initialization schemes ($A \sim \mathcal{N}(0, \sigma^2)$ and $B = 0$) create highly divergent gradient dynamics, spectral velocities, and parameter drift magnitudes. A single scalar threshold cannot govern both spaces.

2. **Subspace Coordinate Variance:** If client variance were computed by projecting updates back to full-rank matrices ($\Delta W = AB$), all computational and memory benefits of PEFT would be lost. Client drift must be measured *directly within the low-rank coordinate subspace.*

3. **Geometric Cross-Contamination:** In alternating optimization, threshold evolution must not leak variance metrics between coordinate tracks. Dynamic thresholds for matrix A must evolve independently from matrix B.

To bridge this divide, this thesis introduces **FDA-RoLoRA**, a unified federated framework that combines alternating coordinate optimization with direct subspace dynamic averaging via a **Decoupled Dual-Track Threshold Mechanism**. By maintaining independent variance monitoring and threshold evolution tracks (θ_A and θ_B), FDA-RoLoRA

achieves simultaneous spatial compression, exact mathematical aggregation linearity, and dynamic communication throttling without projecting updates back to the full-rank parameter space.

To evaluate this approach, we implement an end-to-end multi-process simulation testbed in PyTorch and Hugging Face PEFT, supporting Dirichlet-based non-IID data partitioning across transformer foundation models. Through this framework, we examine the convergence characteristics, communication savings, and parameter sensitivity of dynamic subspace averaging.

1.6 Thesis Outline

The remainder of this diploma thesis is organized as follows:

- **Chapter 2 (Theoretical Background)** establishes the theoretical foundations underpinning this work, reviewing stochastic gradient descent, foundation models, the Transformer architecture, client drift under statistical heterogeneity, server-side adaptive optimizers (FedOpt), Low-Rank Adaptation (LoRA), Robust LoRA (RoLoRA), and Federated Dynamic Averaging (FDA).
- **Chapter 3 (Methodology: FDA-RoLoRA)** presents the mathematical formulation of our proposed approach, detailing the alternating coordinate toggling protocol, direct subspace variance computation, independent halting criteria, dual-track threshold evolution strategies, and server-side adaptive optimization.
- **Chapter 4 (System Architecture and Implementation)** details the software engineering architecture of the simulation platform, covering PyTorch adapter injection, Dirichlet non-IID partitioning engines, multi-process inter-client communication, and cluster execution workflows.
- **Chapter 5 (Experimental Evaluation & Analysis)** defines the benchmarking protocol, comparative baseline algorithms, evaluation metrics across GLUE classification tasks, and presents an in-depth analytical investigation of expected convergence behaviors, communication trade-offs, and threshold dynamics under active empirical evaluation.
- **Chapter 6 (Conclusion and Future Directions)** summarizes the scientific contributions of the thesis, reviews practical engineering limitations, and outlines promising trajectories for future research.

Chapter 2

Background

This chapter provides the foundational concepts necessary to understand the proposed methodology. We begin by reviewing the fundamental principles of neural networks and optimization via gradient descent. We then discuss the architectural scaling of foundation models, the transfer learning paradigm, and the resulting communication bottlenecks inherent in decentralized distributed systems. Next, we examine the core federated learning framework—including client drift under statistical heterogeneity, the Dirichlet partitioning methodology, and server-side adaptive optimization—before quantifying the communication costs that motivate parameter-efficient methods. We then survey the parameter-efficient fine-tuning landscape, focusing on Low-Rank Adaptation and the asymmetric properties of its factor matrices, and analyze the spatial aggregation inconsistency problem that arises when integrating low-rank adapters into federated training. Finally, we explore the geometric monitoring foundations and temporal optimization mechanisms underlying Federated Dynamic Averaging, establishing the theoretical mechanics that underpin FDA-RoLoRA presented in Chapter 3.

2.1 Neural Networks and Optimization

2.1.1 Neural Networks

Supervised learning with neural networks is a cornerstone of modern machine learning, enabling systems to learn complex mappings from input data to desired outputs. In this setting, the objective is to learn a function parameterized by a neural network that generalizes well to unseen data.

In the forward pass, an input vector x is processed through the network layers to produce a prediction. This can be formally denoted as:

$$\hat{y} = f_w(x) \tag{2.1}$$

where f_w represents the neural network parameterized by a set of weights w. The goal of the learning process is to adjust w such that the predictions $\hat{y}$ closely match the true labels y.

To quantify the performance of the network, we define a loss function. For a dataset containing n samples, the empirical risk or average loss $\mathcal{L}(w)$ is computed as:

$$\mathcal{L}(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_w(x_i), y_i) \tag{2.2}$$

where $\ell(\cdot, \cdot)$ is a sample-wise loss function (e.g., cross-entropy for classification). Minimizing this loss function with respect to the parameters w is the central optimization problem in training neural networks.

2.1.2 Gradient Descent

The minimization of the empirical risk is typically performed using gradient-based optimization. The gradients of the loss function with respect to the network weights, denoted as $\nabla_w \mathcal{L}(w)$, are computed efficiently using the backpropagation algorithm. Backpropagation systematically applies the chain rule of calculus from the output layer back to the input layer, distributing the error gradients across all intermediate parameters.

With the gradients computed, the network weights are iteratively updated using Gradient Descent (GD). The fundamental GD update rule at training step t is defined as:

$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t) = w_t - \eta \frac{1}{n} \sum_{i=1}^{n} \nabla_w \ell(f_{w_t}(x_i), y_i) \tag{2.3}$$

where η is the learning rate, a hyperparameter controlling the step size of the update. In standard GD, the true gradient is computed over the entire dataset of size n at each iteration.

2.1.3 Stochastic Gradient Descent

In practice, computing the full gradient over the entire dataset is computationally prohibitive for large datasets. Therefore, Stochastic Gradient Descent (SGD) and its mini-batch variant are universally employed. In mini-batch SGD, the true gradient is approximated using a small subset or mini-batch of m samples (where $m \ll n$). The update rule becomes:

$$w_{t+1} = w_t - \eta \frac{1}{m} \sum_{j=1}^{m} \nabla_w \ell(f_{w_t}(x_{i_j}), y_{i_j}) \tag{2.4}$$

This introduces stochasticity into the optimization process, which not only accelerates each training iteration by reducing the computational cost per step, but can also aid the model in escaping shallow local minima.

While vanilla SGD is effective, modern deep learning often utilizes adaptive optimization algorithms, such as Adam. These optimizers maintain moving averages of past gradients and squared gradients to scale the learning rate on a per-parameter basis, accelerating convergence and improving stability. As we will discuss in later sections, the concepts behind adaptive optimizers are highly relevant when addressing the challenges of server-side optimization in distributed settings.

2.2 Foundation Models and the Transformer Architecture

In recent years, the machine learning paradigm has undergone a profound shift, transitioning from task-specific neural networks to large-scale pre-trained *foundation models* [4]. Foundation models, which include Large Language Models (LLMs) such as GPT-3 [6], RoBERTa [26], and Llama [39], are trained on vast unlabeled datasets using self-supervised

learning, enabling them to be adapted to a broad range of downstream tasks with minimal structural modifications.

At the core of virtually all modern foundation models is the Transformer architecture introduced by Vaswani et al. [40]. Unlike traditional Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs) that process sequential data step-by-step or through localized receptive fields, the Transformer relies fundamentally on the multi-head self-attention mechanism. For an input representation matrix $X \in \mathbb{R}^{n \times d}$, self-attention projects X into Query (Q), Key (K), and Value (V) representations using parameter matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.5}$$

This formulation allows the network to capture pairwise interactions between all tokens in a sequence simultaneously, enabling high computational parallelization during training.

However, the remarkable empirical success of foundation models has been driven by massive parameter scaling. Modern architectures frequently range from hundreds of millions (e.g., RoBERTa-Large with 355M parameters) to tens or hundreds of billions of trainable parameters (e.g., Llama-2 with 7B to 70B parameters) [39]. This unprecedented scale introduces severe computational and memory challenges. Full fine-tuning—updating all weights $w \in \mathbb{R}^d$ during downstream optimization—requires storing activation states, gradients, and optimizer momentum buffers for billions of parameters simultaneously. Consequently, deploying and adapting foundation models shifts machine learning from single-device training to distributed infrastructures and parameter-efficient adaptation paradigms.

The practical utility of foundation models is realized through the *transfer learning* paradigm. During the first phase, a model is pre-trained on a massive, general-purpose corpus using self-supervised objectives (e.g., masked language modeling for encoder models or next-token prediction for autoregressive models), learning broad linguistic and semantic representations. In the second phase, this pre-trained model is *fine-tuned* on a smaller, task-specific labeled dataset—such as sentiment classification, natural language inference, or instruction following [29]—by continuing gradient-based optimization on the downstream objective. The key insight is that the representations learned during pre-training transfer effectively to downstream tasks, enabling strong performance with far less task-specific data than training from scratch. However, standard fine-tuning still requires updating all d parameters, which motivates the parameter-efficient approaches discussed in subsequent sections.

2.3 Distributed Machine Learning

As the complexity of tasks and the volume of available data have grown, both datasets and neural network architectures have become too large to be efficiently processed on a single machine. This computational bottleneck has motivated the development of distributed machine learning systems, where the training process is partitioned across multiple worker nodes or machines. A comprehensive landscape of modern parallel and distributed learning methodologies, including detailed taxonomies of synchronization models and parallelism paradigms, is surveyed by Subasi et al. [35].

Distributed training strategies generally fall into two categories: data parallelism and

model parallelism [35]. In data parallelism, the dataset is divided into partitions distributed across the workers, while each worker maintains a complete replica of the model parameters. Conversely, in model parallelism, a single model is too large to fit in the memory of a single machine, so its layers or components are distributed across multiple workers, while they all process the same data stream. Data parallelism is the most prevalent approach in scenarios scaling across large numbers of nodes.

In a standard data-parallel setup using synchronous SGD, all worker nodes independently compute gradients on their local data batches. Once all workers finish computing their local gradients, they synchronize by aggregating them, typically using an all-reduce operation. The globally averaged gradient is then used to update the model parameters across all nodes simultaneously. While mathematically equivalent to centralized mini-batch SGD with a larger batch size, synchronous SGD suffers from the straggler effect: the entire system must wait for the slowest worker to complete its computation before proceeding to the next step.

To mitigate the straggler problem, asynchronous SGD was introduced, often implemented via a parameter server architecture (illustrated in Figure 2.1). In this paradigm, workers independently fetch the latest model parameters from a central server, compute gradients on their local data, and push the gradients back to the server without waiting for other workers. The server updates the model immediately upon receiving a gradient. However, this lack of synchronization introduces the staleness problem: a worker might compute gradients based on an older version of the model, and by the time it pushes its update, the server's model has already been updated by other workers. These stale gradients can introduce noise, destabilize training, and degrade final model performance.

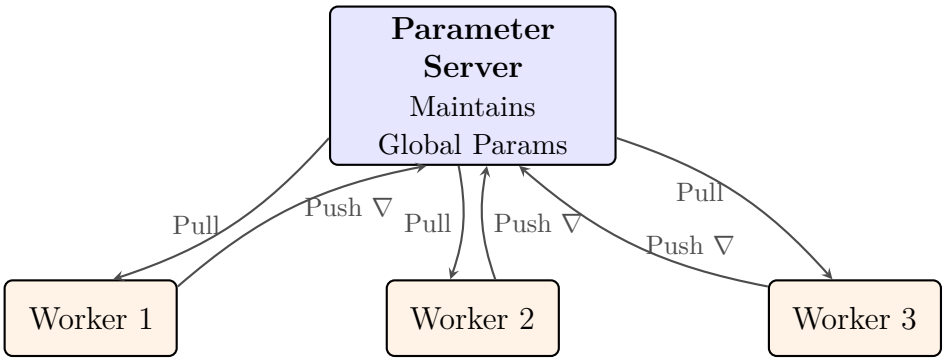


Figure 2.1: Parameter Server Architecture for Asynchronous Distributed Learning. Workers independently pull the latest parameters and push computed gradients.

The evolution of these distributed optimization techniques laid the groundwork for scaling machine learning. However, traditional distributed systems assume reliable communication networks, homogeneous compute nodes, and centrally accessible data. The emergence of privacy concerns and the proliferation of edge devices necessitated a paradigm shift, giving rise to Federated Learning as a privacy-preserving variant of distributed learning that addresses the unique challenges of decentralized data.

2.4 Federated Learning and Communication Bottlenecks

Federated Learning significantly diverges from classical distributed learning by focusing on privacy preservation across decentralized edge devices. Unlike traditional distributed setups with centrally managed data and homogeneous reliable workers, FL must navigate highly heterogeneous local datasets, unreliable network connections, and computationally constrained clients.

2.4.1 Federated Averaging

Federated Learning [17] enables collaborative machine learning across a set of decentralized clients K without centralizing private data. Figure 2.2 illustrates the typical FL lifecycle, comprising local training and global aggregation. Each client $k \in K$ retains its private dataset $\mathcal{D}_k$. The collective objective is to solve the distributed optimization problem:

$$\min_{w \in \mathbb{R}^d} F(w) = \frac{1}{|K|} \sum_{k \in K} F_k(w) \tag{2.6}$$

where $F_k(w) = \mathbb{E}_{x \sim \mathcal{D}_k}[\ell(w; x)]$ represents the local objective function of client k over a loss function $\ell(w; x)$ parameterized by the model weights $w \in \mathbb{R}^d$.

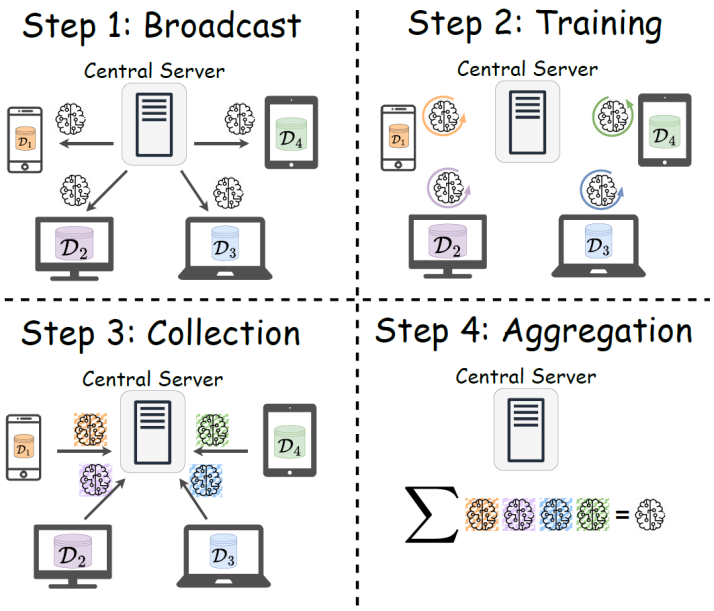


Figure 2.2: Chronological sequence of a Federated Learning communication round (broadcast, local client training, update collection, and server aggregation). Figure reproduced from Theologitis et al. [38].

To solve this problem, the standard Federated Averaging (FedAvg) algorithm [28] operates in communication rounds. At round t, the server selects a client cohort $S_t \subset K$ and broadcasts the global model w_t. The selected clients initialize their local models as $w_{t,0}^{(k)} = w_t$ and perform τ local steps of Stochastic Gradient Descent (SGD):

$$w_{t,j+1}^{(k)} = w_{t,j}^{(k)} - \eta \nabla \ell(w_{t,j}^{(k)}; x) \tag{2.7}$$

After training, clients transmit their local updates $\Delta_t^{(k)} = w_{t,\tau}^{(k)} - w_t$ to the server, which aggregates them to compute the next global model:

$$w_{t+1} = w_t + \frac{1}{|S_t|} \sum_{k \in S_t} \Delta_t^{(k)} \tag{2.8}$$

2.4.2 FedOpt: Server-Side Adaptive Optimization

For modern large foundation models, transmitting full weight vectors $w \in \mathbb{R}^d$ introduces severe spatial communication overhead. Furthermore, client-level statistics are often noisy, and clients are typically stateless, preventing the direct use of adaptive optimizers on edge devices. To address this, the FedOpt framework [30] shifts adaptive optimization to the server. Leveraging the adaptive optimization principles (e.g., Adam) discussed in Section 2.1, FedOpt allows the server to aggregate client parameter displacements $\Delta_t^{(k)} = w_{t,\tau}^{(k)} - w_t$ across the cohort S_t:

$$\Delta_t = \frac{1}{|S_t|} \sum_{k \in S_t} \Delta_t^{(k)} \tag{2.9}$$

Because local updates move along descent trajectories, Δ_t represents the average parameter displacement. In gradient-based server optimizers (e.g., FedAdam, FedAdamW), this quantity defines an effective server pseudo-gradient with an inverted sign:

$$\nabla_{\text{pseudo},t} = -\Delta_t = -\frac{1}{|S_t|} \sum_{k \in S_t} \Delta_t^{(k)} \tag{2.10}$$

ensuring that standard gradient-descent update steps ($w_{t+1} = w_t - \eta_s \nabla_{\text{pseudo},t}$, as formalized in Chapter 3) step in the direction of cumulative local progress, or equivalently through an additive displacement step:

$$w_{t+1} = \text{ServerOpt}(w_t, \Delta_t) \tag{2.11}$$

The motivation for server-side adaptive optimization in federated learning is distinct from its role in centralized training. In centralized settings, adaptive optimizers like Adam [20] accelerate convergence by maintaining per-parameter learning rate scaling. In federated settings, the pseudo-gradient $\nabla_{\text{pseudo},t}$ suffers from compounded noise: it is computed from a sampled client subset $S_t \subset K$ rather than all clients, and each client's local update reflects multiple SGD steps on heterogeneous data rather than a single gradient evaluation. Server-side momentum buffers smooth these noisy pseudo-gradients across rounds, providing a more stable optimization trajectory.

Furthermore, federated clients are inherently *stateless*: they do not persist optimizer state (momentum, variance estimates) between communication rounds, as each round begins with a fresh initialization from the global model. This precludes the use of adaptive optimizers at the client level across rounds, making the server the only entity capable of maintaining long-term optimization history. Additionally, in parameter-efficient adaptation regimes (detailed in Section 2.5), decoupled weight decay (AdamW [27]) at the server provides implicit regularization on adapter parameters, helping prevent overfitting to the heterogeneous local distributions encountered during federated training.

While FedOpt stabilizes training and accelerates convergence, it still relies on rigid, static synchronization schedules (fixed τ), ignoring local optimization dynamics and leading to temporal communication inefficiencies.

2.4.3 Client Drift and Statistical Heterogeneity

A defining challenge in federated learning is the statistical heterogeneity of client data distributions. In practice, client datasets $\mathcal{D}_k$ are rarely independent and identically distributed (IID). Instead, they exhibit various forms of heterogeneity: *label skew*, where different clients possess data from different subsets of classes; *quantity skew*, where the number of local training samples varies significantly across clients; and *feature skew*, where the underlying input distributions differ even for the same label [50].

Under non-IID data, the local objective functions $F_k(w)$ may have significantly different minimizers. When clients perform multiple steps of local SGD on their heterogeneous data, their parameters drift toward distinct local optima that diverge from the global optimum. This phenomenon, known as *client drift* [18], causes the aggregated global update to deviate from the true gradient direction, degrading convergence speed and final model quality.

The severity of client drift is formally analyzed by Li et al. [24], who establish convergence bounds for FedAvg under non-IID data that depend on the degree of gradient dissimilarity across clients. Various drift-mitigation strategies have been proposed: FedProx [23] adds a proximal regularization term to constrain local updates to remain close to the global model, while SCAFFOLD [18] introduces variance reduction through control variates that correct for client drift at each local step. These methods highlight that *monitoring and controlling the divergence between local and global model states is fundamental to stable federated optimization*—a principle that directly motivates the variance-driven synchronization approach explored later in this chapter.

2.4.4 Dirichlet-Based Data Partitioning

To rigorously evaluate federated algorithms under controlled non-IID conditions, the research community has adopted the Dirichlet distribution as the standard tool for synthetic data partitioning [24,50]. For a classification task with C classes and K clients, each client k receives a class proportion vector $p_k \sim \mathrm{Dir}(\alpha)$, where $\mathrm{Dir}(\alpha)$ denotes the symmetric Dirichlet distribution with concentration parameter $\alpha > 0$. Training samples of each class are then allocated to clients according to these sampled proportions.

The concentration parameter α provides a single scalar knob to control the degree of heterogeneity. As $\alpha \to 0$, the sampled proportion vectors become increasingly peaked, producing extreme label skew where individual clients may hold data from only a single class. Conversely, as $\alpha \to \infty$, the proportions converge to a uniform distribution, recovering the IID setting. Intermediate values (e.g., $\alpha = 0.5$ or $\alpha = 1.0$) produce moderate heterogeneity that reflects realistic scenarios where clients have partial but imbalanced class coverage. This configurability makes the Dirichlet partitioning scheme essential for conducting systematic ablation studies across varying degrees of statistical adversity.

2.4.5 Communication Cost Quantification

To make the communication bottleneck concrete, consider the per-round transmission cost for foundation models. For a transformer backbone such as `roberta-base` (125 M parameters), transmitting the full model weights in standard 32-bit floating point precision requires approximately 500 MB per client per synchronization round (or $\sim$ 250 MB in 16-bit half-precision). Across a modest cohort of 10 clients over a 100-round federated training run, this results in up to 500 GB of raw model transfers over uplink-constrained edge

networks. In multi-billion parameter foundation architectures (such as Llama-2-7B with $\approx 7 \times 10^9$ parameters), a single full-model synchronization demands approximately 28 GB in FP32 per client per round, which is completely intractable in distributed federated settings.

To overcome this prohibitive uplink requirement, parameter-efficient fine-tuning (PEFT) methods restrict trainable parameters to compact modular adapters. In particular, Low-Rank Adaptation (LoRA, formalised in Section 2.5) decomposes weight updates into low-rank factor matrices. When targeting the attention projection matrices of `roberta-base` with an adapter rank $r = 8$, the communicated payload drops from 500 MB to approximately 1.2 MB per client per round—achieving a spatial compression ratio exceeding $400\times$. This dramatic reduction demonstrates why low-rank parametrization is essential to make federated training of foundation models feasible across bandwidth-constrained edge networks.

2.5 Parameter-Efficient Fine-Tuning

The massive scale of foundation models renders traditional full fine-tuning—updating all d parameters for a downstream task—increasingly impractical. Full fine-tuning requires storing not only the model weights but also the corresponding activation states, gradient tensors, and optimizer momentum buffers (e.g., first and second moment estimates for Adam) for every trainable parameter. For a model with d parameters in mixed-precision training, the aggregate memory footprint can exceed $16d$ bytes, placing models with hundreds of millions or billions of parameters well beyond the capacity of commodity client hardware. Parameter-Efficient Fine-Tuning (PEFT) addresses this challenge by updating only a small subset of parameters while keeping the vast majority of pre-trained weights frozen.

2.5.1 The PEFT Landscape

Several families of PEFT techniques have been developed to balance parameter efficiency with representational capacity:

- **Adapter Layers**: Introduced by Houlsby et al. [14], adapter modules inject small, learnable bottleneck layers between existing transformer feed-forward and self-attention projections. While parameter-efficient, they introduce structural changes that incur non-negligible inference latency.

- **Prefix Tuning**: Proposed by Li and Liang [25], prefix tuning prepends learnable, continuous task-specific vectors directly to the key and value sequences of transformer attention layers, steering model behavior without modifying original parameters. However, prepending prefixes reduces the effective sequence length available for downstream context.

- **Prompt Tuning**: A further simplification of prefix tuning that prepends virtual token embeddings strictly to the input layer, offering high parameter economy but often exhibiting slower optimization convergence on complex tasks.

2.5.2 Low-Rank Adaptation (LoRA)

Among parameter-efficient adaptation paradigms, Low-Rank Adaptation (LoRA) [16] has emerged as the prevailing standard for both centralized and distributed environments. LoRA operates on the foundational hypothesis that weight updates during downstream task adaptation occupy a low intrinsic rank [1].

Rather than modifying a dense pre-trained weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ directly, LoRA parameterizes the weight update ΔW through a low-rank matrix decomposition:

$$\Delta W = \frac{\alpha}{r} BA \tag{2.12}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ denotes the down-projection matrix, $B \in \mathbb{R}^{d_{\text{out}} \times r}$ denotes the up-projection matrix, rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$, and α is a constant scaling hyperparameter. The forward pass of a modified layer computes:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx \tag{2.13}$$

Throughout fine-tuning, the pre-trained backbone weights W_0 remain completely frozen, receiving zero gradient updates. Only the compact factor matrices A and B are optimized. Under the canonical initialization scheme, A is initialized with random Gaussian values $\mathcal{N}(0, \sigma^2)$, while B is initialized to zero, ensuring that $\Delta W = \frac{\alpha}{r} BA = 0$ at training onset and the model's pre-trained predictions are exactly preserved prior to adaptation. The scaling factor $\frac{\alpha}{r}$ stabilizes the optimization dynamics when varying the adapter rank r, typically set such that $\alpha = 2r$. Figure 2.3 illustrates this architecture.

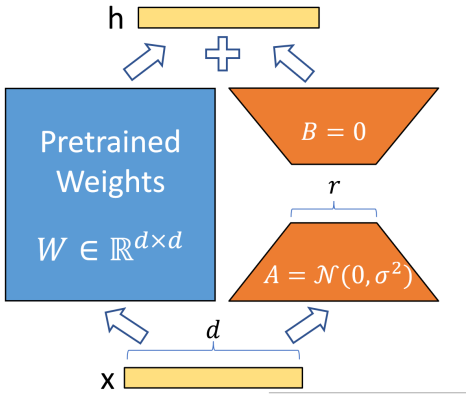


Figure 2.3: Low-Rank Adaptation (LoRA) schematic: decomposition of weight updates into low-rank matrices A and B alongside frozen pretrained weights W.

Unlike adapter layers, LoRA introduces zero additional inference latency: upon completion of training, the factorized update $\frac{\alpha}{r} BA$ can be folded directly into the pre-trained weights ($W_{\text{final}} = W_0 + \frac{\alpha}{r} BA$) via simple matrix addition. In centralized settings, extensions such as QLoRA [9] (which integrates 4-bit NormalFloat base quantization) and AdaLoRA [48] (which dynamically allocates rank budgets across layers) have further highlighted the versatility of the low-rank paradigm.

2.5.3 Structural and Dynamic Asymmetry in LoRA Matrices

An essential, frequently overlooked property of the LoRA decomposition is the inherent asymmetry between its two factor matrices. Functionally, the down-projection matrix $A \in \mathbb{R}^{r \times d_{\text{in}}}$ compresses input features from high-dimensional representations into the low-rank bottleneck, whereas the up-projection matrix $B \in \mathbb{R}^{d_{\text{out}} \times r}$ expands compressed representations back to the original dimensionality. These distinct projection roles produce asymmetric gradient dynamics during backpropagation.

Furthermore, the canonical initialization scheme creates an acute temporal asymmetry: because $B = 0$ at initialization, the initial gradients with respect to A ($\nabla_A \mathcal{L} \propto B^\top \nabla_h \mathcal{L}$) are initially zero, whereas the gradients with respect to B ($\nabla_B \mathcal{L} \propto \nabla_h \mathcal{L}(Ax)^\top$) immediately receive non-zero projections from the Gaussian-initialized A. Consequently, the optimization trajectories, convergence velocities, and parameter variance profiles of A and B diverge substantially during early training rounds, with matrix B typically manifesting significantly higher drift. In federated learning, this asymmetry implies that treating both coordinate matrices identically during aggregation or synchronization scheduling is fundamentally suboptimal.

2.6 Federated Low-Rank Adaptation

Having established standard LoRA and the core FL framework, combining the two paradigms—formalized as Federated Low-Rank Adaptation (FedLoRA) [45]—is a natural evolution to reduce the massive communication costs of federated learning. In a typical FL setting, transmitting full model updates $w \in \mathbb{R}^d$ imposes severe network strain. By adopting LoRA, clients only train and transmit the low-rank matrices A and B, which drastically reduces the spatial communication overhead of federated synchronization.

A comprehensive survey of FedLoRA [45] categorizes the primary challenges in this domain into distributed learning stability, data heterogeneity, and efficiency. To address these challenges, several frameworks have been proposed. For instance, FedIT [47] applies standard federated averaging directly to LoRA parameters. To mitigate communication costs further and ensure exact updates, FFA-LoRA [36] freezes the down-projection matrices A across all clients, allowing clients to update and transmit only the up-projection matrices B. Other approaches, such as FLoRA [43], support heterogeneous-rank adapters among clients to accommodate varying device capacities.

Despite its benefits, aggregating independent low-rank matrices from clients introduces a critical issue known as *LoRA Aggregation Discordance* [45]. When standard federated averaging (FedAvg) is applied separately to the client-specific matrices A_i and B_i, it introduces a fundamental mathematical inconsistency:

$$\frac{1}{N}\sum_{i=1}^{N}(B_i A_i) \neq \left(\frac{1}{N}\sum_{i=1}^{N} B_i\right)\left(\frac{1}{N}\sum_{i=1}^{N} A_i\right) \tag{2.14}$$

This inequality demonstrates that independent averaging of B_i and A_i fails to reconstruct the true average of the weight updates due to the non-linear, bilinear nature of matrix multiplication. As a result, applying FedAvg directly to these factorized matrices leads to inaccurate global aggregation and severe model degradation.

To address this discordance, the literature proposes several alternative strategies [45]. One approach is full-size matrix aggregation, where clients reconstruct the full $\Delta W =$

$\frac{\alpha}{r}BA$ before transmission; however, this entirely negates the spatial communication benefits of LoRA. Another approach involves SVD recalculation, where the server aggregates full matrices and then performs Singular Value Decomposition to extract new low-rank factors, which introduces prohibitive computational overhead at the server. Compared to these computationally or communication-heavy alternatives, alternating single-matrix aggregation has emerged as a highly efficient and structurally sound solution.

2.7 Robust Low-Rank Adaptation

While LoRA successfully reduces the spatial communication cost, the aggregation inconsistency described above poses a fundamental barrier to its deployment in federated settings. RoLoRA addresses this without sacrificing adapter expressiveness.

To resolve this inconsistency without limiting adapter expressiveness, RoLoRA [7] introduces an alternating optimization and aggregation framework. Instead of permanently freezing one of the low-rank matrices, RoLoRA alternates the active training coordinate space across communication rounds.

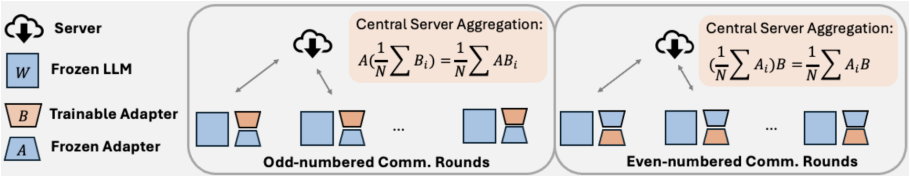


Figure 2.4: Robust Low-Rank Adaptation (RoLoRA) alternating optimization scheme. In each round, one low-rank matrix is frozen globally while the other is trained and aggregated linearly.

Note on notation: While Chen et al. [7] designate their low-rank product as AB by naming the up-projection A and down-projection B, we maintain the standard LoRA convention $\Delta W = \frac{\alpha}{r}BA$ [16] throughout this thesis to preserve mathematical consistency with common PEFT architectures and the formal framework developed in Chapter 3.

In odd-numbered communication rounds $(2t+1)$, the down-projection matrix A^t is globally frozen and shared across all clients i, while only the up-projection matrix B is trained locally. The server then aggregates these updates:

$$B^{t+1} = \frac{1}{N}\sum_{i=1}^{N} B_i^{t+1} \tag{2.15}$$

Because A^t is identical across all clients, the aggregation is strictly linear, introducing no spatial interference:

$$\frac{1}{N}\sum_{i=1}^{N}(B_i^{t+1}A^t) = \left(\frac{1}{N}\sum_{i=1}^{N} B_i^{t+1}\right)A^t = B^{t+1}A^t \tag{2.16}$$

In even-numbered communication rounds $(2t+2)$, the aggregated up-projection matrix B^{t+1} is frozen, while the clients update the down-projection matrix A locally. The server aggregates these updates:

$$A^{t+1} = \frac{1}{N}\sum_{i=1}^{N} A_i^{t+1} \tag{2.17}$$

Since B^{t+1} is now identical across all clients, linearity is preserved:

$$\frac{1}{N}\sum_{i=1}^{N}(B^{t+1}A_i^{t+1}) = B^{t+1}\left(\frac{1}{N}\sum_{i=1}^{N}A_i^{t+1}\right) = B^{t+1}A^{t+1} \tag{2.18}$$

This alternating optimization enables full parameter space exploration while maintaining zero-interference linear aggregations throughout the federated training process.

2.8 Federated Dynamic Averaging

Having resolved the spatial communication bottleneck through RoLoRA's linearized aggregation, we now address the temporal dimension: controlling the synchronization frequency based on actual training dynamics rather than a fixed schedule.

Federated Dynamic Averaging (FDA) and its optimization variant FDA-Opt [38] were developed to optimize the training schedule dynamically. FDA is rooted in the principles of Geometric Monitoring [33], a framework originally developed for threshold function monitoring over distributed data streams.

2.8.1 Geometric Monitoring Foundations

The core insight of geometric monitoring is that a global condition over distributed data can be decomposed into local conditions that each node monitors independently. Consider N distributed nodes, each holding a local state vector $v_i(t)$ that evolves over time. The global state is the average $\bar{v}(t) = \frac{1}{N}\sum_{i=1}^{N} v_i(t)$. The monitoring objective is to detect when a threshold function $f(\bar{v}(t))$ crosses a predefined boundary, without requiring continuous global synchronization.

Geometric monitoring achieves this through the concept of *safe zones*. Each node i defines a bounding region around a reference point e_i (typically the last synchronized global estimate). As long as the local drift vector $\delta_i(t) = v_i(t) - e_i$ remains within its safe zone, the global condition is guaranteed to hold by a convexity argument: if all local states lie within a convex safe zone centered around the global reference, their average must also lie within that zone. Crucially, this means that *no communication is necessary* as long as every node's local constraint is satisfied. A synchronization round is triggered only when at least one node's drift exits its safe zone, indicating that the global condition may have been violated.

FDA adapts this distributed monitoring framework to the federated learning setting. The "state vectors" become local model parameter updates, the "threshold function" becomes model variance (measuring client drift divergence), and the "safe zone" is defined by a variance budget θ. When local parameter drift across clients causes the variance to exceed θ, synchronization is triggered. This reformulation transforms the rigid, schedule-based synchronization of standard FL into an event-driven protocol that communicates only when training dynamics demand it.

For a sampled subset of clients S_t at communication round t, let $\Delta_{t,i}^{(k)} = w_{t,i}^{(k)} - w_{t,0}$ denote the local update of client k at step i relative to the initial global model $w_{t,0}$. The average update (pseudo-gradient) across all clients is defined as:

$$g_{t,i} = \frac{1}{|S_t|}\sum_{k\in S_t}\Delta_{t,i}^{(k)} \tag{2.19}$$

The model variance quantifies the dispersion of local client updates in the parameter space and is defined as:

$$\mathrm{Var}(W_{t,i}) = \frac{1}{|S_t|} \sum_{k \in S_t} \left\| \Delta_{t,i}^{(k)} \right\|_2^2 - \|g_{t,i}\|_2^2 \tag{2.20}$$

Here, the first term measures the average local training progress across clients, while the second term captures the magnitude of the aggregated global step. Consequently, a low model variance value signifies either negligible local gradient progress or, more favorably, highly aligned local updates across participating clients where local optimization trajectories coincide. Conversely, a high variance reading manifests when clients achieve substantial local parameter movement along divergent optimization vectors towards conflicting local minima, signaling the onset of severe client drift.

During local training, clients track model variance. Computing this exact variance requires full-dimensional vector operations, which can be computationally expensive and communication-heavy to track continuously in a distributed setting. To address this, the original FDA protocol utilizes the Alon-Matias-Szegedy (AMS) sketch [2], a randomized algorithm that efficiently approximates the L_2 norm (and thus the variance) using minimal memory and sub-linear space. While full-parameter models require randomized sketching to approximate variance in sub-linear memory, we show in Chapter 3 that restricting geometric monitoring to compact low-rank coordinate tensors allows exact variance evaluation with negligible computational and memory overhead, rendering probabilistic sketching unnecessary in federated LoRA regimes.

Synchronization is triggered immediately if the variance exceeds a dynamic threshold: $\mathrm{Var}(W_{t,i}) > \theta_t$. Because the variance magnitude evolves by orders of magnitude from early to late training stages, FDA-Opt dynamically scales this threshold across rounds using a linear prediction based on the previous round's termination step s_t:

$$\theta_{t+1} = \mathrm{ar}_t \cdot \frac{\tau}{2s_t} \tag{2.21}$$

where ar_t is the actual variance at termination in round t, s_t is the termination step, and τ is the target training duration. This linear threshold prediction assumes that standard training variance typically grows linearly and monotonically during a round. Furthermore, to eliminate the synchronization overhead of checking variance at every training step, queries are restricted to a sparse subset of steps (typically once per epoch), mitigating the communication bottleneck of distributed geometric checks.

While RoLoRA resolves the spatial aggregation inconsistency and FDA addresses the temporal communication cadence, combining the two paradigms requires careful algorithmic formulation. In particular, low-rank parameter trajectories exhibit heightened variance volatility, and alternating optimization demands decoupled threshold management. To address these challenges, Chapter 3 formalizes **FDA-RoLoRA** to combine dynamic averaging with alternating low-rank adaptation, explaining how it internally deploys a dual-track threshold mechanism to track subspace divergence independently. The concrete execution loops of both baseline protocols are formally summarized in Section 3.2 before introducing our synthesized methodology.

Chapter 3

Methodology

The research presented in this thesis investigates the integration of Federated Dynamic Averaging (FDA) [38] with Parameter-Efficient Low-Rank Adaptation (LoRA) [16]. By bridging dynamic variance-driven synchronization with low-rank parameter factorizations, this work aims to jointly optimize the spatial and temporal dimensions of federated learning.

3.1 Combining Dynamic Averaging with Federated LoRA

Existing literature in Parameter-Efficient Federated Fine-Tuning focuses almost exclusively on spatial parameter compression, while universally relying on static Federated Averaging (FedAvg) [28] for temporal synchronization. Under standard FedAvg, all clients execute a predetermined, fixed number of local epochs ($\tau = E$) regardless of local optimization drift, data heterogeneity, or gradient volatility.

The concept of integrating dynamic averaging into federated low-rank architectures envisions replacing rigid, static FedAvg schedules with adaptive, variance-driven averaging. Crucially, combining FDA with different federated LoRA variants cannot be treated as a single, one-size-fits-all formula. Because distinct PEFT methods manipulate low-rank matrices through fundamentally different mechanisms—ranging from simultaneous factor updates to frozen projections, sparse allocations, or alternating coordinates—each pairing suggests a distinct architectural design:

- **FDA + FedLoRA**: Explores dynamic synchronization alongside standard simultaneous training of low-rank matrices A and B. In this configuration, dynamic averaging could monitor the joint parametric drift or product-space variance of the complete adapter. While this pairing retains the characteristic bilinear aggregation of standard FedLoRA ($\frac{1}{N}\sum A_i B_i \neq (\frac{1}{N}\sum A_i)(\frac{1}{N}\sum B_i)$), it represents the most direct, unconstrained synthesis of dynamic scheduling and low-rank adaptation.

- **FDA + FFA-LoRA**: Explores dynamic averaging under frozen down-projections [36], where matrix A remains fixed after random initialization. Because parameter evolution is restricted solely to the up-projection B, dynamic variance monitoring simplifies to tracking a single active subspace coordinate space.

- **FDA + SLoRA**: Explores dynamic synchronization tailored to sparse low-rank

parameter representations [3], where threshold adaptation and variance estimation could be structured around coordinate-sparse adapter allocations across clients.

- **FDA + RoLoRA (Our investigation)**: Explores dynamic averaging in conjunction with alternating coordinate optimization [7]. By alternating active updates between down-projection matrix A and up-projection matrix B across global communication rounds, this avenue naturally isolates each subspace, preserving update linearity ($\frac{1}{N}\sum AB_i = A(\frac{1}{N}\sum B_i)$) and enabling independent, decoupled threshold tracking (θ_A, θ_B).

Among these architectural possibilities, we select **FDA + RoLoRA** as our focal investigation due to the unique mathematical advantages of coordinate alternation. By isolating updates to a single low-rank factor per communication round, RoLoRA provides strict linearity in parameter aggregation, eliminating inter-factor interference. This structural property provides an exceptionally clean theoretical foundation for formulating decoupled subspace variance tracking with independent geometric thresholds. We consider the broader taxonomy—such as joint variance tracking in simultaneous FedLoRA or coordinate-sparse thresholding in SLoRA—as promising avenues for future research.

The key insight enabling this integration is that FDA's variance monitoring can be applied independently to each subspace, as RoLoRA's alternating scheme naturally isolates the optimization trajectories of the down-projection (A) and up-projection (B) matrices. To regulate threshold updates across communication rounds, the framework adopts a responsive, parameter-free midpoint-scaling threshold update rule that dynamically adapts synchronization intervals to local client drift.

3.2 Foundational Precursor Protocols: RoLoRA and FDA

To rigorously contextualize the design of FDA-RoLoRA, we examine the operational execution loops of its two constituent foundational frameworks: Federated Dynamic Averaging (FDA) [38] and Robust Low-Rank Adaptation (RoLoRA) [7]. While both methods address fundamental inefficiencies in decentralized learning, they tackle orthogonal dimensions of the federated optimization bottleneck:

- **FDA (Temporal Optimization, Full-Rank Model Space)**: Evaluates the dispersion of local client updates in parameter space. By deploying geometric safe zones via the AMS sketch, FDA replaces static round schedules with variance-triggered synchronization, dynamically terminating local training at $e^* \leq E$ when client drift exceeds a scalar threshold θ. However, the standard protocol assumes simultaneous parameter updates over the full model weight space, leaving low-rank factorized architectures unaddressed.

- **RoLoRA (Spatial Optimization, Fixed Local Schedule)**: Eliminates bilinear aggregation inconsistency by alternating coordinate optimization between low-rank factor matrices (A and B) across global communication rounds. When factor B is trained, down-projection A is frozen, preserving update linearity ($\frac{1}{N}\sum A_i B = (\frac{1}{N}\sum A_i)B$). However, RoLoRA enforces a rigid, fixed local epoch schedule (E) across all clients and rounds, leaving temporal synchronization unoptimized.

Algorithm 1 and Algorithm 2 summarize the algorithmic structures of FDA and RoLoRA side-by-side.

Algorithm 1: FDA

Input: Clients K, cohort $|S_r|$, initial θ_0, limit E

Initialize global parameters W_0;
for *round* $r = 0, 1, 2, \ldots$ **do**
  Broadcast $W_0^{(r)}$ to cohort S_r;
  for *epoch* $e = 1$ **to** E **do**
    foreach *client* $i \in S_r$ ***in*** ***parallel*** **do**
      $W_{i,e}^{(r)} \leftarrow W_{i,e-1}^{(r)} - \eta_c \nabla \mathcal{L}_i$;
      $\Delta_i^{(e)} = W_{i,e}^{(r)} - W_0^{(r)}$;
    Evaluate AMS variance;
    if $Var(\Delta^{(e)}) > \theta_r$ **then**
      $e^* \leftarrow e$; **break**;
  $W_0^{(r+1)} \leftarrow \frac{1}{|S_r|} \sum_{i \in S_r} W_{i,e^*}^{(r)}$;
  Update scalar threshold θ_{r+1};

Algorithm 2: RoLoRA

Input: Clients K, cohort $|S_r|$, fixed epoch limit E

Initialize $W = W_{pretrained} + A_0 B_0$;
for *round* $r = 0, 1, 2, \ldots$ **do**
  Select active $\mathcal{M}^{(r)} \in \{B, A\}$;
  Freeze complementary matrix;
  Broadcast active $W_{act}^{(r)}$ to S_r;
  for *epoch* $e = 1$ **to** E **do**
    foreach *client* $i \in S_r$ ***in*** ***parallel*** **do**
      Train $\mathcal{M}^{(r)}$ on local data;
      $\Delta_i = W_{i,e}^{(r)} - W_{act}^{(r)}$;
  Collect client drifts $\{\Delta_i\}_{i \in S_r}$;
  $\Delta_{agg} \leftarrow \frac{1}{|S_r|} \sum_{i \in S_r} \Delta_i$;
  $W_{act}^{(r+1)} \leftarrow W_{act}^{(r)} + \Delta_{agg}$;

Contrasting these two protocols reveals the central architectural gap: RoLoRA provides the necessary update linearity to eliminate spatial distortion, but lacks adaptive temporal control; FDA provides dynamic temporal adaptivity, but cannot handle coordinate-isolated, asymmetric low-rank subspaces with a single scalar threshold. This limitation directly motivates our proposed **FDA-RoLoRA**, which synthesizes alternating coordinate isolation with decoupled subspace variance tracking via an internal dual-track threshold mechanism.

3.3 Notation and Hyperparameters

Consider a global communication round indexed by r, during which an active matrix $\mathcal{M}^{(r)} \in \{A, B\}$ is designated for client-side optimization. Within round r, local client optimization progresses through discrete local epoch iterations $e \in \mathbb{N}$, culminating in a final realized local epoch count $e^* \in \mathbb{N}$ at the instant synchronization is triggered. At any given evaluation epoch e, each client i computes its raw subspace parametric drift $\Delta_i^{(e)}$ relative to the starting global parameters. Rather than projecting these parameter updates back to the expanded model weight space, the framework evaluates the direct subspace variance $\mathrm{Var}_{subspace}(\Delta^{(e)})$ exclusively on the low-rank coordinate tensors of the active matrix.

The operational boundaries and adaptive behavior of FDA-RoLoRA are governed by a set of global hyperparameters. Local client training is bounded between a minimum epoch floor $e_{min} \in \mathbb{N}$ and a maximum epoch ceiling $e_{max} \in \mathbb{N}$. To accommodate the distinct drift dynamics of alternating low-rank factor matrices, FDA-RoLoRA internally implements a dual-track threshold mechanism: synchronization timing is controlled by two persistent, decoupled variance thresholds $\theta_A, \theta_B \in \mathbb{R}^+$, from which the active threshold $\theta_{current}^{(r)}$ is dynamically mapped to match $\mathcal{M}^{(r)}$. Threshold adaptation across rounds is governed by the direct midpoint-scaling update. A comprehensive summary of mathematical symbols, tensor shapes, and hyperparameters is provided in Table 3.1.

Table 3.1: Summary of mathematical notation and system hyperparameters.

Symbol	Domain / Dimensions	Description
$W_{pretrained}$	$\mathbb{R}^{d_{out} \times d_{in}}$	Frozen backbone model parameter weights
r_{rank}	$\mathbb{N}$ $(r_{rank} \ll \min(d_{in}, d_{out}))$	Intrinsic low-rank adapter rank dimension
A	$\mathbb{R}^{r_{rank} \times d_{in}}$	Low-rank down-projection factor matrix
B	$\mathbb{R}^{d_{out} \times r_{rank}}$	Low-rank up-projection factor matrix
K	$\mathbb{N}^+$	Total number of decentralized client nodes in the federation
S_r	$S_r \subseteq \{1, \dots, K\}$	Sampled cohort of participating clients in round r
r	$\mathbb{N}_0 = \{0, 1, 2, \dots\}$	Global communication round index
$\mathcal{M}^{(r)}$	$\{B, A\}$	Designated active parameter matrix in round r
$W_{act}^{(r)}$	Active matrix tensor	Global parameters of the active matrix broadcast to cohort S_r
e	$\{1, \dots, e_{max}\}$	Local training epoch index within a round
e_{min}, e_{max}	$\mathbb{N}^+$ $(e_{min} \leq e_{max})$	Minimum epoch floor and maximum epoch ceiling
e^*	$[e_{min}, e_{max}]$	Realized local epoch count at synchronization trigger
$\Delta_i^{(e)}$	Matching $\mathcal{M}^{(r)}$	Subspace parametric drift of client i at epoch e
$\mathrm{Var}_{subspace}$	$\mathbb{R}_{\geq 0}$	Subspace drift variance across cohort S_r (Eq. 3.5)
θ_A, θ_B	$\mathbb{R}^+$	Persistent, decoupled variance thresholds for tracks A and B
$\theta_{current}^{(r)}$	$\mathbb{R}^+$	Active threshold loaded for round r (θ_A or θ_B)
$\theta_{target}^{(r)}$	$\mathbb{R}^+$	Target threshold proposal computed via midpoint scaling
$\nabla_{pseudo}^{(r)}$	Matching $\mathcal{M}^{(r)}$	Aggregated server-side pseudo-gradient (Eq. 3.8)
η_c, η_s	$\mathbb{R}^+$	Local client learning rate and global server learning rate

Algorithm 3: FDA-RoLoRA

Input: Clients K, cohort size $|S_r|$, server learning rate η_s, local learning rate η_c, min/max epochs e_{min}, e_{max}, initial thresholds θ_A, θ_B

Initialize global parameters $W = W_{pretrained} + A_0 B_0$;
Initialize persistent thresholds θ_A, θ_B and decoupled server optimizer states (for matrices A and B);
Set first-round calibration flags: $\text{FirstRound}_A \leftarrow \text{True}, \text{FirstRound}_B \leftarrow \text{True}$;
for *round* $r = 0, 1, 2, \ldots$ **do**
    Determine active matrix $\mathcal{M}^{(r)} \in \{B, A\}$ (alternate B-first each round);
    Load active threshold $\theta^{(r)}_{current} \leftarrow \theta_{\mathcal{M}^{(r)}}$ and active server optimizer;
    Sample client cohort $S_r \subseteq \{1, \ldots, K\}$;
    Server broadcasts active parameters $W^{(r)}_{act}$ to sampled cohort S_r;
    for *local epoch* $e = 1$ **to** e_{max} **do**
        foreach *client* $i \in S_r$ ***in parallel*** **do**
            Train locally for one epoch on private data: $W^{(r)}_{i,e} \leftarrow W^{(r)}_{i,e-1} - \eta_c \nabla \mathcal{L}_i(W^{(r)}_{i,e-1})$;
            Compute subspace drift: $\Delta^{(e)}_i = W^{(r)}_{i,e} - W^{(r)}_{act}$;
        if $e \geq e_{min}$ **then**
            Server collects drifts $\{\Delta^{(e)}_i\}_{i \in S_r}$ and evaluates cohort variance $\text{Var}_{subspace}(\Delta^{(e)})$ via Eq. 3.5;
            if $\mathit{FirstRound}_{\mathcal{M}^{(r)}}$ **then**
                $e^* \leftarrow e$;
                $\text{FirstRound}_{\mathcal{M}^{(r)}} \leftarrow \text{False}$;
                break;
            if $\mathit{Var}_{subspace}(\Delta^{(e)}) > \theta^{(r)}_{current}$ ***or*** $e = e_{max}$ **then**
                $e^* \leftarrow e$;
                break;
    Server computes pseudo-gradient $\nabla^{(r)}_{pseudo} = -\frac{1}{|S_r|} \sum_{i \in S_r} \Delta^{(e^*)}_i$ via Eq. 3.8;
    Update active matrix threshold $\theta_{\mathcal{M}^{(r)}} \leftarrow \theta^{(r)}_{target}$ via midpoint scaling (Eq. 3.9), keeping inactive threshold frozen;
    Apply active server optimizer (Gradient Descent / FedAvg with η_s) to update $W^{(r+1)}_{act}$ via Eq. 3.10;

3.4 FDA-RoLoRA Initialization and Subspace Toggling

At the start of round r, the server determines the active target matrix $\mathcal{M}^{(r)} \in \{B, A\}$ adhering to a B-first alternating schedule. Internally, FDA-RoLoRA operates a dual-track threshold mechanism that maintains independent variance tracking for each coordinate space. To govern the local synchronization dynamics, the system loads the persistent threshold corresponding exclusively to the active matrix:

$$\theta_{current}^{(r)} = \begin{cases} \theta_A & \text{if } \mathcal{M}^{(r)} = A \\ \theta_B & \text{if } \mathcal{M}^{(r)} = B \end{cases} \tag{3.1}$$

The active weight matrix is initialized across all sampled clients $i \in S_r$ using the global model parameters broadcast by the server:

$$W_{i,0}^{(r)} = W_{act}^{(r)} \tag{3.2}$$

3.5 Local Training and Direct Subspace Variance

Clients advance local parameters using independent Stochastic Gradient Descent [31] steps. For client i at local step t, the parameter update operates on the active parameter tensor:

$$W_{i,t}^{(r)} = W_{i,t-1}^{(r)} - \eta_c \nabla \mathcal{L}_i(W_{i,t-1}^{(r)}) \tag{3.3}$$

where η_c is the local learning rate and $\mathcal{L}_i$ is the client's empirical loss.

Local training proceeds epoch-by-epoch. At each epoch boundary $e \geq e_{min}$, participating clients compute their raw subspace parametric drift relative to the starting global parameter state:

$$\Delta_i^{(e)} = W_{i,e}^{(r)} - W_{act}^{(r)} \tag{3.4}$$

To minimize communication overhead, exact variance is computed directly on the isolated low-rank coordinates of the active matrix, avoiding projection to the expanded parameter space:

$$\mathrm{Var}_{subspace}(\Delta^{(e)}) = \frac{1}{|S_r|} \sum_{i \in S_r} \left\| \Delta_i^{(e)} \right\|_2^2 - \left\| \frac{1}{|S_r|} \sum_{i \in S_r} \Delta_i^{(e)} \right\|_2^2 \tag{3.5}$$

Crucially, performing this variance check at intermediate epoch boundaries ($e \in [e_{min}, e_{max}]$) does not impose a prohibitive communication burden on the federated network. In contemporary foundation models (such as RoBERTa-Large or Llama-2-7B), low-rank adapter factor matrices ($r_{rank} \in \{8, 16\}$) represent less than 1%—and frequently under 0.2%—of the total backbone parameter footprint, yielding a greater than 99% parameter reduction relative to full-rank fine-tuning. Because the coordinates $\Delta_i^{(e)}$ are strictly confined to the active low-rank tensor (A or B), transmitting these low-dimensional drifts to the server to evaluate cohort dispersion incurs negligible bandwidth overhead compared to standard full-model parameter exchanges. Consequently, exact low-rank variance tracking provides fine-grained temporal monitoring without the communication bottlenecks typically associated with frequent synchronization probing.

3.6 Server-Coordinated Synchronization Halt and Calibration

At each epoch boundary $e \geq e_{min}$, the server gathers the low-rank client drifts $\{\Delta_i^{(e)}\}_{i \in S_r}$ to evaluate the collective cohort variance $\mathrm{Var}_{subspace}(\Delta^{(e)})$. The server then determines whether to trigger global aggregation or instruct clients to proceed to the subsequent local epoch:

$$\text{Halt Local Training and Trigger Aggregation if } \mathrm{Var}_{subspace}(\Delta^{(e)}) > \theta_{current}^{(r)} \quad \vee \quad e \geq e_{max} \tag{3.6}$$

If the drift variance remains within the threshold and $e < e_{max}$, clients continue local optimization without model aggregation. The final realized local epoch count at synchronization is recorded as $e^* \in [e_{min}, e_{max}]$.

To eliminate manual tuning of initial variance thresholds, the protocol incorporates an empirical bootstrap calibration mechanism during the very first activation of each coordinate track (round 0 for matrix B, round 1 for matrix A). In these initial rounds, training automatically halts at $e^* = e_{min}$. The server measures the baseline drift variance $\mathrm{Var}_{subspace}(\Delta^{(e_{min})})$ and calibrates the starting threshold via Eq. 3.7:

$$\theta_{\mathcal{M}^{(r)}} \leftarrow \left(\frac{e_{max}/2}{e_{min}}\right) \mathrm{Var}_{subspace}(\Delta^{(e_{min})}) \tag{3.7}$$

Subsequent rounds for that coordinate track then evolve dynamically using the calibrated baseline.

3.7 Global Aggregation and Independent Threshold Evolution

Upon receiving the client updates, the server aggregates the subspace drifts to compute the pseudo-gradient:

$$\nabla_{pseudo}^{(r)} = -\frac{1}{|S_r|} \sum_{i \in S_r} \Delta_i^{(e^*)} \tag{3.8}$$

The server then updates the active matrix parameters. Crucially, the variance threshold for the active matrix $\mathcal{M}^{(r)}$ is updated independently to preserve state for its next training cycle, while the threshold for the inactive matrix remains frozen.

We define the target threshold update via midpoint scaling. The active threshold is adjusted based on the realized epoch count e^* relative to the nominal training midpoint $e_{max}/2$:

$$\theta_{\mathcal{M}^{(r)}}^{(next)} = \left(\frac{e_{max}/2}{e^*}\right) \mathrm{Var}_{subspace}(\Delta^{(e^*)}) \tag{3.9}$$

This direct update rule establishes a self-stabilizing negative feedback loop:

- When client updates diverge rapidly due to local gradient volatility or data heterogeneity, the variance threshold is breached early ($e^* < e_{max}/2$). Consequently, the scaling multiplier $(e_{max}/2)/e^* > 1$ increases the threshold tolerance for the subsequent round of that coordinate track, preventing premature synchronization halts.

- Conversely, when client trajectories remain aligned and stable ($e^* > e_{max}/2$), the scaling multiplier is strictly less than 1, systematically tightening the threshold to encourage timely synchronization before excessive local client drift accumulates.

Crucially, this adaptation is completely parameter-free, requiring no historical window tuning or heuristic hyperparameter calibration.

3.7.1 Numerical Walkthrough of Dual-Track Evolution

To illustrate the operational dynamics of decoupled dual-track threshold evolution in practice, consider an illustrative pedagogical scenario with example hyperparameter bounds $e_{min} = 2$ and $e_{max} = 10$, setting the target midpoint to $e_{max}/2 = 5$:

1. **Round 0 ($\mathcal{M}^{(0)} = B$, Bootstrap Calibration):**
   Being the first activation of coordinate track B, local training is halted automatically at $e^* = e_{min} = 2$. The server evaluates the baseline cohort drift variance via Eq. 3.5, yielding $\mathrm{Var}_{subspace}(\Delta^{(2)}) = 0.040$. The initial threshold for track B is calibrated as:

   $$\theta_B \leftarrow \left(\frac{5}{2}\right) \times 0.040 = 2.5 \times 0.040 = 0.100$$

   Track A remains uncalibrated, with its calibration flag active.

2. **Round 1 ($\mathcal{M}^{(1)} = A$, Bootstrap Calibration):**
   The active matrix toggles to A. The calibrated threshold $\theta_B = 0.100$ is preserved in memory and frozen. Track A executes its bootstrap round, halting at $e^* = e_{min} = 2$. With a measured baseline variance of $\mathrm{Var}_{subspace}(\Delta^{(2)}) = 0.024$, track A calibrates its independent threshold:

   $$\theta_A \leftarrow \left(\frac{5}{2}\right) \times 0.024 = 2.5 \times 0.024 = 0.060$$

3. **Round 2 ($\mathcal{M}^{(2)} = B$, Dynamic Adaptive Round):**
   The active matrix returns to B, loading $\theta^{(2)}_{current} = \theta_B = 0.100$, while $\theta_A = 0.060$ remains frozen. At local epoch $e = 2$, the server measures cohort variance at $0.052 \leq 0.100$; local optimization continues. At epoch $e = 3$, heterogeneous local updates push the drift variance to $\mathrm{Var}_{subspace}(\Delta^{(3)}) = 0.108 > 0.100$. Synchronization triggers immediately at $e^* = 3$. The server updates track B's threshold via Eq. 3.9:

   $$\theta_B \leftarrow \left(\frac{5}{3}\right) \times 0.108 \approx 1.667 \times 0.108 = 0.180$$

   Track B stores $\theta_B = 0.180$ for its next activation (in Round 4). Throughout Round 2, $\theta_A = 0.060$ remained completely unaffected.

This walkthrough demonstrates how each low-rank subspace tracks its own parameter drift velocity independently, ensuring that gradient scale differences between up-projection and down-projection matrices do not distort synchronization timing.

3.8 Server Optimization and Pseudo-Gradient Dynamics

Following the evaluation of local subspace drifts, the server aggregates client progress to perform the global model update. This aggregation is achieved by computing an explicit pseudo-gradient $\nabla_{pseudo}^{(r)} = -\frac{1}{|S_r|}\sum_{i\in S_r}\Delta_i^{(e^*)}$ (as defined in Eq. 3.8) and applying it to update the global parameters. This formulation allows the server update to interface seamlessly with generalized server-side optimization algorithms.

In our production benchmark experiments, this pipeline is configured using standard Gradient Descent (SGD) with server learning rate η_s:

$$W_{act}^{(r+1)} = W_{act}^{(r)} - \eta_s \nabla_{pseudo}^{(r)} = W_{act}^{(r)} + \eta_s \left(\frac{1}{|S_r|} \sum_{i\in S_r} \Delta_i^{(e^*)} \right) \tag{3.10}$$

Setting the server optimizer to SGD with learning rate $\eta_s = 1.0$ collapses the update directly to:

$$W_{act}^{(r+1)} = \frac{1}{|S_r|} \sum_{i\in S_r} W_{i,e^*}^{(r)} \tag{3.11}$$

which exactly replicates classical Federated Averaging (FedAvg) over the active low-rank coordinate subspace. In our experimental evaluations, pairing server-side SGD with client-level SGD (using learning rates such as $\eta_c = 0.1$) was chosen to ensure that empirical gains stem strictly from dynamic variance-driven synchronization intervals rather than confounding momentum or preconditioning artifacts. Nevertheless, the complete pseudo-gradient optimizer infrastructure remains implemented in the codebase and fully functional, ready to be explored with adaptive optimizers in future investigations.

From an optimization perspective, adopting vanilla Gradient Descent on the server side intentionally neutralizes and mitigates the theoretical mechanisms of adaptive federated optimization [30]. In generalized FedOpt frameworks, casting local drifts as pseudo-gradients is motivated by the desire to leverage adaptive algorithms (such as FedAdam or FedYogi) that maintain running first moments (m) to smooth out noisy stochastic drift from partial client participation, alongside second moments (v) to provide coordinate-wise learning rate scaling ($\frac{1}{\sqrt{v}+\epsilon}$). However, several considerations favor plain gradient descent in alternating low-rank regimes. First, vanilla gradient descent eliminates the need for tuning additional sensitive server-side hyperparameters ($\beta_1, \beta_2, \epsilon$), providing hyperparameter parsimony across disparate datasets and partitioning regimes. Second, because plain SGD computes no second moments ($v_t \equiv 0$), all low-rank coordinate dimensions are updated with equal step size, mitigating the risk that noisy, sparsely updated low-rank dimensions produce unstable scaling factors in the denominator. Finally, in alternating optimization under FDA-RoLoRA, parameter blocks A and B toggle across rounds; maintaining historical momentum buffers across interrupted optimization passes can cause stale gradient signals to distort updates when a coordinate block reactivates. Direct linear aggregation via SGD eliminates this momentum inertia, ensuring that each global update reflects only the immediate consensus of the current training round.

For distributed environments exhibiting extreme non-IID client heterogeneity where adaptive curvature compensation is advantageous, the FDA-RoLoRA architecture retains full modular compatibility with adaptive optimizers. To prevent momentum staleness and cross-matrix contamination, the server maintains two strictly isolated optimizer states—

(m_A, v_A) for matrix A and (m_B, v_B) for matrix B.

Let $m_{\mathcal{M}}$ and $v_{\mathcal{M}}$ denote the active moment buffers. When employing standard Adam [20] on the server, the moment estimates and parameter updates are formulated as:

$$m_{\mathcal{M}}^{(r)} = \beta_1 m_{\mathcal{M}}^{(r-1)} + (1-\beta_1)\nabla_{pseudo}^{(r)} \tag{3.12}$$

$$v_{\mathcal{M}}^{(r)} = \beta_2 v_{\mathcal{M}}^{(r-1)} + (1-\beta_2)\left(\nabla_{pseudo}^{(r)}\right)^2 \tag{3.13}$$

$$\hat{m}_{\mathcal{M}}^{(r)} = \frac{m_{\mathcal{M}}^{(r)}}{1-\beta_1^r}, \quad \hat{v}_{\mathcal{M}}^{(r)} = \frac{v_{\mathcal{M}}^{(r)}}{1-\beta_2^r} \tag{3.14}$$

$$W_{act}^{(r+1)} = W_{act}^{(r)} - \eta_s \frac{\hat{m}_{\mathcal{M}}^{(r)}}{\sqrt{\hat{v}_{\mathcal{M}}^{(r)}} + \epsilon} \tag{3.15}$$

Alternatively, when incorporating decoupled weight decay [27], the AdamW server optimizer modifies the parameter update step as:

$$W_{act}^{(r+1)} = W_{act}^{(r)} - \eta_s \left(\frac{\hat{m}_{\mathcal{M}}^{(r)}}{\sqrt{\hat{v}_{\mathcal{M}}^{(r)}} + \epsilon} + \lambda W_{act}^{(r)} \right) \tag{3.16}$$

where η_s is the server learning rate, β_1, β_2 are moment decay rates, ϵ is a numerical stability constant, and λ is the weight decay coefficient.

In summary, FDA-RoLoRA provides a communication-efficient strategy for federated fine-tuning, demonstrating how dynamic averaging can be synthesized with alternating low-rank adaptation through an internal dual-track threshold mechanism. By systematically decoupling low-rank subspace updates and regulating synchronization frequency through independent, dynamically scaled variance thresholds, the methodology resolves the geometric and temporal challenges of distributed LoRA training. The following Implementation chapter will detail the software architecture, the integration of Hugging Face PEFT, and the practical realization of this theoretical model within the experimental testbed.

Chapter 4

Implementation

The software architecture developed for this thesis realizes the integration of Federated Dynamic Averaging with parameter-efficient fine-tuning via FDA-RoLoRA, utilizing an internal dual-track threshold mechanism to manage coordinate subspace variance, open-sourced under the **FDA-RoLoRA** repository. This chapter details the software architecture, the strategies for data partitioning across clients, the configuration of the targeted foundation models, the mechanics of the training pipeline, and the high-performance computing environment utilized for experimental evaluation.

4.1 Software Architecture

The framework is developed in Python, utilizing the PyTorch deep learning framework as the foundational library for automatic differentiation and tensor operations. The codebase is built as a direct extension of the FDA-Opt repository [38],[1] inheriting its robust core federated orchestration infrastructure. To implement the adaptation strategies, the system heavily relies on the Hugging Face Transformers library in conjunction with the Parameter-Efficient Fine-Tuning (PEFT) library, which has established itself as the research standard for LoRA-based methods. The complete executable software architecture developed for this thesis has been open-sourced as **FDA-RoLoRA**.[2]

A key practical constraint faced during this research was the absence of a publicly available code repository for RoLoRA [7], which only noted using the FederatedScope-LLM platform [21]. Given that we had direct access to the FDA-Opt codebase [38], the design choice was between porting FDA's dynamic variance monitoring and threshold evolution logic into the heavy, event-driven architecture of FederatedScope-LLM, or implementing RoLoRA's alternating optimization protocol within the existing FDA-Opt framework. We chose the latter approach, as implementing RoLoRA inside the lightweight FDA-Opt core was substantially more tractable and transparent. This decision afforded complete architectural visibility and direct control over low-rank tensor serialization and threshold updates, without the abstraction overhead of a third-party framework.

To manage the complexities of federated learning, the system follows a clear three-tier architecture comprising a Server Orchestrator, Client Nodes, and a Communication Interface. The Server Orchestrator assumes a centralized role; it maintains the global LoRA parameter matrices A and B, tracks the current threshold states θ_A and θ_B for synchronization, manages server-side optimizer momentum buffers, and increments a global round

[1] https://github.com/michaeltheologitis/FDA-Opt
[2] https://github.com/Asterinos1/FDA-RoLoRA

counter responsible for the RoLoRA alternating optimization toggle. The Client Nodes represent the edge devices or data silos. During each communication round, a subset of sampled clients receives the active parameter block (A or B), performs local SGD on their private datasets across dynamic local epoch intervals, and returns the accumulated parameter drift vectors (Δ_i) to the server, which centrally evaluates cohort variance and orchestrates the aggregation step. Finally, the Communication Interface orchestrates the logical data exchange between clients and the server. In our simulation testbed, client-server communication is modeled in-silico: rather than incurring physical network socket overhead, parameter drift vectors (Δ_i) are calculated via direct in-memory tensor subtractions against the round baseline snapshot. Communication bandwidth consumption is evaluated analytically by measuring the exact serialized byte footprint of the active low-rank coordinate tensors transmitted per client per round (e.g., 4.69 MB for rank $r = 4$ adapters in single-precision float32). Unlike conventional federated learning frameworks that transmit full model weights, this design yields a massive reduction in bandwidth requirements.

Crucially, the alternating optimization protocol—the toggling mechanism inherent to the RoLoRA methodology—is strictly managed server-side. This centralization guarantees global consistency across all participating clients, ensuring that during any given round, every active client is optimizing and communicating updates for the same LoRA matrix block.

4.2 Data Partitioning

The software architecture implements client data partitioning following the Dirichlet allocation protocol defined in Section 2.4.4. During environment initialization, the partitioning module loads the targeted GLUE corpus. To address the significant disparity in scale across benchmark tasks (ranging from under 70,000 training examples in SST-2 to nearly 400,000 in MNLI) and mitigate prohibitive simulation runtimes on the shared GPU cluster, the framework adopts an empirical dataset ceiling. Specifically, the training volume is capped at an upper ceiling of 25,000 examples via deterministic pseudo-random subsampling, while smaller corpora and all validation evaluation sets are preserved in full. Preliminary pilot investigations confirmed that this sample budget provides sufficient linguistic diversity and gradient support for adapter fine-tuning without compromising convergence stability or task performance, while keeping multi-seed simulation horizons computationally tractable. Following this volume standardization, the module segments the active corpus into K isolated local partitions according to the configured Dirichlet concentration parameter α. Each simulated client receives a dedicated local partition and batch iterator, enforcing strict data isolation during local training without cross-client leakage.

4.3 Simulation Controls and Early Stopping

Beyond dataset scale, substantial wall-clock latency accumulates during simulation because client cohorts ($C \in \{20, 50\}$) are evaluated sequentially on the GPU to respect accelerator memory limits. To prevent excessive simulation runtimes across multi-seed sweeps and avoid computing redundant rounds on converged models, the framework incorporates an **accuracy-gated early stopping controller** into the Server Orchestrator.

At the conclusion of each communication round, the server evaluates the consensus model on the centralized task validation split. The controller governs early termination via four complementary mechanisms:

1. **Trailing Sliding-Window Smoothing (W)**: In federated optimization under statistical heterogeneity, validation trajectories exhibit stochastic oscillations due to client sampling and non-IID drift. To prevent premature halts triggered by transient dips, the controller tracks a trailing moving average of validation accuracy across a sliding window of W rounds.

2. **Accuracy Activation Gate (A_{gate})**: Because alternating coordinate optimization requires several initial rounds to calibrate both factor matrices (A and B), early stopping is strictly disarmed during early training. The patience counter activates only after the smoothed validation accuracy crosses a predefined milestone threshold (A_{gate}).

3. **Patience and Minimum Improvement ($\Delta_{\min}, P$)**: Once the activation gate is breached, the controller monitors validation progress over a patience window of P consecutive rounds. If the moving-average accuracy fails to improve by at least $\Delta_{\min} = 0.5\%$, the training loop terminates cleanly ahead of $R_{\max}$.

4. **Mandatory Round Guard ($R_{\min}$)**: To ensure that models explore sufficient alternating coordinate cycles regardless of metric volatility, the engine enforces a strict round floor ($R_{\min}$) before any termination signal can trigger.

This controller functions as an external simulation management harness, accelerating cluster turnaround times without modifying the underlying federated optimization dynamics. The specific hyperparameter configurations deployed across each GLUE benchmark task and partitioning regime are detailed in Section 5.1.3.

4.4 Model Configuration

The experimental implementation targets transformer-based foundation models. In this work, the empirical evaluation focuses specifically on `roberta-base` [26] (comprising approximately 125 million parameters), evaluated on sequence classification tasks from the GLUE benchmark [41]. While the original RoLoRA paper [7] evaluated primarily on `roberta-large` (355M parameters), we deliberately chose `roberta-base` to accommodate our institutional hardware budget (2× NVIDIA A10 GPUs) and project turnaround time limits. This choice cuts VRAM allocation and backward pass latency by more than half, enabling thorough multi-seed sweeps across 20–50 clients while maintaining representative transformer attention geometry. For `roberta-base`, the low-rank adapters are systematically injected into the query and value projection matrices (W_q and W_v) within each self-attention layer.

Across the model configuration, the LoRA rank is strictly constrained to $r = 4$ (and $r = 8$), accompanied by scaling factor $\alpha_{\text{lora}} = 8$ (and $\alpha_{\text{lora}} = 16$) to maintain activation magnitudes. During the federated fine-tuning process, all original pretrained foundation model weights as well as the downstream sequence classification head are kept completely frozen. The only trainable parameters across all communication rounds are the low-rank adapter matrices A and B. This setup provides a profound advantage in communication

cost, as communicating only the compact low-rank drift matrices rather than the full 125M parameter set yields orders-of-magnitude reductions in client-server bandwidth requirements. While broader foundation models (such as RoBERTa-Large or autoregressive decoders like Llama-2-7B) represent natural targets for future scaled deployment, our implementation and empirical study focus on `roberta-base` across GLUE tasks.

4.5 Training Pipeline

The operational training pipeline translates the theoretical FDA-RoLoRA protocol (formulated in Algorithm 3) into an efficient, stateful PyTorch execution loop. At the start of communication round t, the Server Orchestrator determines the active parameter track by evaluating round parity ($t \pmod 2$), activating track B on even rounds and track A on odd rounds. To enforce alternating coordinate isolation in software, the orchestrator traverses the model parameter tree (`model.named_parameters()`) and dynamically toggles the PyTorch autograd flags: parameters matching the active adapter key are assigned `requires_grad = True`, while the dormant adapter block, the downstream classification head, and all backbone foundation model parameters are assigned `requires_grad = False`. The server snapshots the active parameters ($W_{\text{act}}^{(0)}$) as the baseline for tracking parametric drift, binds the corresponding independent server optimizer instance, loads the active threshold state (θ_B or θ_A), and synchronizes the active weights to the sampled client cohort $S_t \subset \{1, \ldots, K\}$. Maintaining two persistent `torch.optim` instances across rounds keeps optimizer momentum states strictly partitioned, preventing cross-phase state contamination.

Within each simulated communication round, sampled clients are evaluated sequentially on the allocated GPU accelerator, executing local SGD over their private data partitions using a configurable local learning rate η_c and local batch size B. This sequential simulation design faithfully models the logical decentralized execution of independent client nodes while strictly constraining GPU VRAM footprint to a single active client instance at any given instant. Because inactive weights have autograd disabled, PyTorch allocates gradient and optimizer memory exclusively for the active low-rank coordinates, minimizing client VRAM consumption. Local training proceeds across discrete epoch boundaries bounded by runtime-configurable limits $[e_{\min}, e_{\max}]$ (typically configured within $[1, 20]$ across our empirical foundation model benchmarks). For each epoch $e \geq e_{\min}$, the framework extracts each client's local parameter drift $\Delta_i^{(e)} = W_{i,e} - W_{\text{act}}^{(0)}$ via direct in-memory tensor subtraction against the round baseline. The orchestrator flattens each drift tensor, stacks them via `torch.stack`, and computes the exact subspace variance $\text{Var}_{\text{subspace}}(e)$ across the cohort following Eq. 3.5 in single-digit milliseconds using vectorized PyTorch operations (`torch.norm`). The loop automatically halts at $e^* = e_{\min}$ during the first round of each matrix track to record baseline variance and initialize the threshold via Eq. 3.7. In subsequent rounds, the server dynamically halts local training at the earliest epoch e^* where $\text{Var}_{\text{subspace}}(e) > \theta_{\text{current}}$, or upon reaching the ceiling $e^* = e_{\max}$ (Eq. 3.6).

Following inner loop termination, the server aggregates the gathered drifts into the pseudo-gradient $G_t = -\frac{1}{|S_t|} \sum_{i \in S_t} \Delta_i^{(e^*)}$ (Eq. 3.8). Rather than performing manual weight substitution, the server restores active parameters to $W_{\text{act}}^{(0)}$, injects G_t directly into the parameter `.grad` buffers, and invokes `optimizer.step()` using the active server optimizer instance. With standard SGD ($\eta_s = 1.0$), this operation executes coordinate-wise Federated Averaging (Eq. 3.11) while preserving a decoupled interface compatible with

adaptive server optimizers (such as AdamW, Eq. 3.16). The server then recalibrates the active track's synchronization threshold using the Standard midpoint scaling rule (Eq. 3.9) while keeping the dormant threshold frozen in memory.

To prevent accelerator memory fragmentation across hundreds of simulation rounds on the A10 GPUs, the pipeline enforces strict memory hygiene: gradient buffers are deallocated via `optimizer.zero_grad(set_to_none=True)`, intermediate computation graphs and drift tensors are explicitly detached, and the orchestrator periodically flushes the CUDA cache using `torch.cuda.empty_cache()` before incrementing the round counter ($t \leftarrow t + 1$) to launch the complementary matrix phase.

4.6 Deployment Environment

The development and execution environment for the FDA-RoLoRA framework is designed to ensure strict software reproducibility and computational efficiency. The software stack is standardized across all nodes,[3] pinning core libraries to guarantee consistent autograd graph mechanics, deterministic low-rank tensor serialization, and reproducible pseudo-gradient aggregation across simulation runs.

Production experiments are executed on the Polytechnix server of the SoftNet Lab at the Technical University of Crete. The server is equipped with enterprise-grade NVIDIA accelerators (2× NVIDIA Ampere A10 GPUs with 24 GB VRAM each), 2× Intel Xeon Silver 4310 CPUs (24 cores / 48 threads total at 2.10 GHz), and 256 GB of RAM, providing the necessary computational throughput for fine-tuning foundation models.

A suite of custom orchestration bash and Python scripts manages the execution lifecycle. Rather than spawning parallel client processes within a single simulation (which would saturate GPU VRAM), multi-process parallelism is leveraged at the cluster level: the orchestration scripts dispatch distinct experimental configurations (spanning multiple random seeds, baseline comparisons, and Dirichlet partitioning regimes) concurrently across the two available NVIDIA A10 GPUs. Within each running simulation job, the script coordinates dynamic synchronization rounds, logs tracking metrics, and automates the extensive parameter sweeps required for empirical evaluation and ablation analysis.

[3] All experiments were conducted using Python 3.10.12, PyTorch 2.5.1+cu121, Hugging Face Transformers 5.5.4, and PEFT 0.19.1, running CUDA 12.1 with cuDNN 9.1 on Ubuntu Linux.

Chapter 5

Evaluation

This chapter presents the empirical evaluation framework, experimental methodology, and benchmark results designed to assess the efficacy of the proposed **FDA-RoLoRA** framework (which operates an internal dual-track threshold mechanism to decouple coordinate-space variance) alongside comparative baselines.

Our empirical investigations target transformer-based foundation models under statistical data heterogeneity, focusing specifically on the `roberta-base` architecture across sequence classification tasks from the standard General Language Understanding Evaluation (GLUE) benchmark suite. Full-scale federated foundation model fine-tuning was executed on our high-performance computing testbed. This chapter outlines the foundation model benchmarking methodology, details the baseline implementations, analyzes empirical convergence and stability dynamics, and reports the completed benchmark results across GLUE sequence classification tasks.

5.1 Experimental Setup for Foundation Models

All empirical benchmarks in this chapter are conducted using the software framework and foundation model adaptation pipeline established in Chapter 4, executed on the high-performance computing environment described in Section 4.6. In accordance with the model and adapter configuration detailed in Section 4.4, our empirical evaluation targets the `roberta-base` foundation model (125M parameters) with rank $r \in \{4, 8\}$ adapters injected into the query and value attention projections (W_q and W_v).

This section specifies the downstream benchmark tasks and corpus standardization policy (Section 5.1.1), the Dirichlet statistical data partitioning settings (Section 5.1.2), and the accuracy-gated early stopping protocol deployed to govern communication round horizons (Section 5.1.3).

5.1.1 Benchmark Tasks and Corpus Standardization

We evaluate performance on the General Language Understanding Evaluation (GLUE) benchmark suite [41], selecting four sequence classification tasks spanning distinct linguistic domains and corpus scales: Stanford Sentiment Treebank (SST-2) [34], Quora Question Pairs (QQP), Question Natural Language Inference (QNLI), and Multi-Genre Natural Language Inference (MNLI) [44].

To standardize compute throughput across disparate task scales and bound simulation runtimes on the GPU cluster, we enforce the deterministic dataset ceiling introduced

in Section 4.2. Specifically, the maximum training volume is strictly bounded to 25,000 examples across all evaluated benchmarks. Datasets exceeding this threshold (MNLI with 392,702 sentence pairs, QQP with 363,846 pairs, QNLI with 104,743 pairs, and SST-2 with 67,349 sentences) are deterministically sub-sampled to 25,000 training examples prior to client partitioning, while smaller corpora and validation evaluation sets are preserved in full.

5.1.2 Statistical Data Partitioning

Following the Dirichlet allocation protocol defined in Section 2.4.4, client datasets are partitioned across two empirical settings:

- **Homogeneous Setting** ($\alpha = 1000.0$): Mimics an ideal IID distribution with uniform class representation across all clients, evaluating baseline optimization efficiency under consensus gradients.

- **Heterogeneous Setting** ($\alpha = 0.5$): Enforces severe non-IID label skew following RoLoRA [7], where individual clients hold heavily imbalanced class subsets, inducing acute inter-client gradient divergence.

Experiments evaluate client cohorts of size $C \in \{20, 50\}$ under full cohort participation. Across all benchmarks, local client optimization utilizes SGD with learning rate $\eta_c = 0.1$ and local batch size $B = 16$, while server aggregation applies SGD with $\eta_s = 1.0$. Local epoch budgets operate within dynamic boundaries $[e_{\min}, e_{\max}] = [1, 20]$ with cold-start initial thresholds $\theta_A = \theta_B = 0.0$ calibrated via standard midpoint scaling. The dynamic local epoch target is parameterized by $k \in \{0.5, 1.0\}$, where canonical midpoint scaling (Eq. 3.9) corresponds to $k = 0.5$ (targeting $e^* \approx e_{\max}/2$), while $k = 1.0$ evaluates an aggressive setting targeting the upper boundary $e^* \approx e_{\max}$. All evaluations are conducted across multiple random seeds, reporting the mean and standard deviation of each metric to guarantee statistical reliability.

5.1.3 Accuracy-Gated Early Stopping Protocol

To regulate communication round horizons across our empirical sweeps, we deploy the early stopping controller established in Section 4.3. Experiments enforce a maximum budget of $R_{\max} = 50$ rounds for $C = 20$ clients and $R_{\max} = 30$ rounds for $C = 50$ clients, armed once the consensus model crosses task-specific milestone accuracy gates (85% for SST-2, 80% for QNLI, 72% for QQP, and 55–60% for MNLI) with a minimum improvement threshold of $\Delta_{\min} = 0.5\%$.

In homogeneous (IID) settings, where training exhibits smooth, monotonic convergence, tight smoothing and responsive patience ($W = 3, P = 3, R_{\min} = 6$) are configured to promptly terminate redundant rounds once performance plateaus. Conversely, under statistical heterogeneity (Non-IID, $\alpha = 0.5$), we configure looser settings—widening the smoothing window and patience ($W = 5, P = 5$) and enforcing an extended 60% round floor ($R_{\min} = \lfloor 0.6 \times R_{\max} \rfloor$, i.e., 30 rounds for $C = 20$ and 18 rounds for $C = 50$)—to accommodate higher metric volatility and prevent premature termination caused by transient client drift oscillations.

5.2 Comparative Baselines

Our empirical evaluation primarily benchmarks the proposed **FDA-RoLoRA** framework against **RoLoRA** under identical alternating coordinate optimization, directly isolating the temporal communication speedup achieved through variance-governed local synchronization. Furthermore, following the benchmark design of the original RoLoRA paper [7], we also evaluate **FFA-LoRA** and **FedLoRA** as key reference baselines to situate performance within the broader parameter-efficient federated learning landscape:

1. **RoLoRA (Fixed Schedule)**: Primary baseline [7]. Alternates updates between factor matrices A and B, but enforces a rigid local epoch schedule ($E \in \{10, 20\}$).

2. **FDA-RoLoRA (Ours)**: Proposed framework. Couples alternating coordinate optimization with dynamic local epoch selection ($e \in [1, 20]$) governed by dual-track midpoint threshold scaling.

3. **FFA-LoRA**: Structural baseline [36]. Evaluated as in Chen et al. [7], FFA-LoRA permanently freezes the down-projection matrix A upon random initialization and trains only the up-projection matrix B under fixed local epochs ($E = 10$).

4. **FedLoRA**: Conventional baseline [16,28]. Included following the RoLoRA study [7] as the standard federated LoRA benchmark, where both low-rank matrices A and B are updated simultaneously under fixed local epochs ($E = 10$), serving as the uncorrected reference vulnerable to bilinear aggregation inconsistency.

5.3 Foundation Model Evaluation and Empirical Benchmarks

To rigorously assess the performance of the proposed FDA-RoLoRA protocol on full-scale transformer foundation models, we evaluate sequence classification on the GLUE benchmark suite using `roberta-base` (125M parameters) across SST-2, MNLI, and QNLI. Table 5.1 summarizes the comparative performance metrics across evaluated baselines and the proposed FDA-RoLoRA framework.

Methodology	SST-2 Acc. (%)	MNLI Acc. (%)	QNLI Acc. (%)	Comm. Rounds	Uplink Payload / Client
Untuned Base (Zero-Shot)	50.90 ± 0.00	35.40 ± 0.00	49.50 ± 0.00	–	0.0 MB
FedLoRA ($r = 4$)	90.48 ± 0.00	76.85 ± 0.90	87.31 ± 0.00	23–45	Full ($2 \times d \times r$)
RoLoRA (Fixed E10)	$\mathbf{91.51 \pm 0.23}$	$\mathbf{74.11 \pm 0.22}$	$\mathbf{81.10 \pm 3.05}$	15–50	**Half** ($1 \times d \times r$)
FFA-LoRA ($r = 4$)	88.10 ± 0.00	69.14 ± 0.00	78.50 ± 0.00	30–50	Half ($1 \times d \times r$)
FDA-RoLoRA (Ours)	91.20 ± 0.40	75.58 ± 0.86	$\mathbf{83.38 \pm 0.00}$	**19–38**	**Half** ($1 \times d \times r$)

Table 5.1: Comparative performance metrics on foundation model benchmarks (`roberta-base` on GLUE tasks, $C = 20$ clients) across evaluated baselines and the proposed FDA-RoLoRA framework.

5.3.1 Homogeneous Benchmark Suite (IID Settings, $\alpha = 1000.0$)

In homogeneous federated learning environments, we configure the Dirichlet concentration parameter to $\alpha = 1000.0$ to mimic an exact IID data distribution across clients. As

outlined in Section 5.1.1, the greater the concentration parameter α, the more homogeneous the client partitions become, as the variance of sampled class proportions vanishes asymptotically toward uniform coverage. With $\alpha = 1000.0$, inter-client label variance is practically eliminated, ensuring that training data is uniformly distributed across client partitions and local empirical distributions closely mirror the global data distribution. Consequently, local gradient trajectories calculated across client cohorts align in coordinate space toward the shared global optimum. This setting provides an ideal testbed for evaluating whether dynamic synchronization can safely expand local computation budgets without inducing client drift.

Table 5.2 reports the comprehensive empirical benchmarks across all four GLUE sequence classification tasks (QQP, QNLI, MNLI, and SST-2) under homogeneous partitioning, comparing the primary fixed-schedule baseline (**RoLoRA**, $E = 10$) against our proposed **FDA-RoLoRA** across both client cohort scales ($C = 20$ and $C = 50$).

GLUE Task	Client Scale	Methodology	Final Acc. (%)	Peak Acc. (%)	Mean Rounds	Cumulative Uplink (MB)
SST-2	$C = 20$ ($R_{\max} = 50$)	RoLoRA ($E = 10$)	**91.51 ± 0.23**	**91.80 ± 0.06**	15.0 ± 0.0	70.3 MB
		FDA-RoLoRA (Ours)	91.20 ± 0.40	91.63 ± 0.20	17.0 ± 1.0	79.7 MB
	$C = 50$ ($R_{\max} = 30$)	RoLoRA ($E = 10$)	89.91 ± 0.34	90.54 ± 0.52	22.0 ± 0.0	257.8 MB
		FDA-RoLoRA (Ours)	**90.15 ± 0.35**	**90.71 ± 0.30**	**19.0 ± 1.0**	**222.7** MB
QQP	$C = 20$ ($R_{\max} = 50$)	RoLoRA ($E = 10$)	75.47 ± 0.84	75.47 ± 0.84	**6.5 ± 0.7**	**30.5** MB
		FDA-RoLoRA (Ours)	**76.34 ± 3.77**	**78.57 ± 2.99**	17.5 ± 3.2	82.0 MB
	$C = 50$ ($R_{\max} = 30$)	RoLoRA ($E = 10$)	**75.43 ± 0.76**	**78.79 ± 0.97**	30.0 ± 0.0	351.6 MB
		FDA-RoLoRA (Ours)	72.65 ± 0.83	73.63 ± 0.02	**14.0 ± 0.0**	**164.1** MB
QNLI	$C = 20$ ($R_{\max} = 50$)	RoLoRA ($E = 10$)	**75.03 ± 7.45**	81.83 ± 0.65	**10.0 ± 0.0**	**46.9** MB
		FDA-RoLoRA (Ours)	67.63 ± 15.75	**83.14 ± 0.24**	16.5 ± 2.5	77.3 MB
	$C = 50$ ($R_{\max} = 30$)	RoLoRA ($E = 10$)	64.56 ± 14.37	65.03 ± 14.65	27.8 ± 4.4	325.8 MB
		FDA-RoLoRA (Ours)	**82.47 ± 1.22**	**82.77 ± 1.51**	**21.0 ± 4.0**	**246.1** MB
MNLI	$C = 20$ ($R_{\max} = 50$)	RoLoRA ($E = 10$)	**76.60 ± 0.60**	**76.60 ± 0.60**	**28.0 ± 0.0**	**131.3** MB
		FDA-RoLoRA (Ours)	75.58 ± 0.86	75.75 ± 0.70	29.5 ± 0.5	138.3 MB
	$C = 50$ ($R_{\max} = 30$)	RoLoRA ($E = 10$)	65.85 ± 0.50	65.85 ± 0.50	30.0 ± 0.0	351.6 MB
		FDA-RoLoRA (Ours)	**65.88 ± 5.60**	**65.88 ± 5.60**	**27.0 ± 3.0**	**316.4** MB

Table 5.2: Empirical evaluation on the homogeneous benchmark suite (IID, Dirichlet $\alpha = 1000.0$) across GLUE sequence classification tasks on `roberta-base`.

The empirical results in Table 5.2 demonstrate three primary behaviors characteristic of the homogeneous setting:

1. **Equivalence Under Small Cohorts ($C = 20$)**: When client data is identically distributed ($\alpha = 1000.0$) and the participating cohort is small ($C = 20$), inter-client parameter variance is negligible. In this setting, clients compute near-consensus gradients, allowing fixed local schedules ($E = 10$) to train safely without encountering client drift. Consequently, fixed RoLoRA and FDA-RoLoRA perform comparably on SST-2 (91.51% vs. 91.20%) and MNLI (76.60% vs. 75.58%), while fixed RoLoRA halts earlier on QQP (6.5 vs. 17.5 rounds). Under these ideal conditions, dynamic variance tracking provides minimal additional leverage because drift variance remains persistently low.

2. **Cohort Scalability and Robustness ($C = 50$)**: The distinct advantage of variance-governed synchronization emerges when scaling the client cohort to $C = 50$. Even with homogeneous data partitions, aggregating across 50 clients amplifies cumulative coordinate divergence. Under fixed $E = 10$, baseline RoLoRA experiences severe stability degradation on QNLI, collapsing to 64.56% final accuracy (65.03% peak). In contrast, FDA-RoLoRA dynamically regulates local epoch budgets to prevent

divergence, maintaining 82.47% final accuracy (82.77% peak)—a +17.91% improvement—while saving 6.8 communication rounds (21.0 vs. 27.8) and reducing uplink bandwidth by 79.7 MB (246.1 MB vs. 325.8 MB). Furthermore, on QQP ($C = 50$), FDA-RoLoRA reaches convergence in only 14.0 rounds (164.1 MB) compared to RoLoRA's 30.0 rounds (351.6 MB), cutting total uplink communication by over 53%.

3. **Preservation of Model Expressivity**: Across all four GLUE benchmarks, alternating linear coordinate aggregation completely avoids the severe bilinear aggregation error of naive FedLoRA, confirming that alternating update serialization preserves full adapter capacity under homogeneous distributed optimization.

5.3.2 Statistical Heterogeneity Benchmark Suite (Non-IID Settings, Dirichlet Skew $\alpha = 0.5$)

While homogeneous settings validate peak optimization efficiency, practical federated networks are characterized by severe statistical data heterogeneity. To rigorously evaluate the resilience and closed-loop control dynamics of our framework, we investigate real-world non-IID conditions modeled via Dirichlet label distribution skew with concentration parameter $\alpha = 0.5$. Adopting $\alpha = 0.5$ directly follows the non-IID evaluation protocol of RoLoRA [7] alongside established non-IID federated benchmarking conventions [15, 50], representing a standard benchmark for severe label distribution skew.

Due to the substantial computational cost of simulating sequential clients over extended non-IID round horizons on our compute cluster, our statistical heterogeneity evaluation focuses specifically on QNLI (binary sentence-pair inference) and MNLI (3-class natural language inference). These tasks represent the most demanding linguistic reasoning benchmarks in our suite, where Dirichlet label skew induces severe gradient conflict across client cohorts: on 3-class MNLI (`entailment`, `neutral`, `contradiction`), individual clients receive disproportionately heavy concentrations of a single class while remaining almost entirely deficient in others. On 2-class QNLI, clients observe heavily biased binary class ratios. When clients train locally on skewed data, their parameter updates diverge sharply in coordinate space, inducing acute inter-client drift.

Table 5.3 summarizes the empirical performance metrics under Dirichlet label skew ($\alpha = 0.5$) across QNLI and MNLI, comparing fixed-budget RoLoRA baselines ($E = 10$ and $E = 20$) against our adaptive FDA-RoLoRA configurations across epoch bounds and scaling parameters k.

GLUE Task	Client Scale	Methodology	Final Acc. (%)	Peak Acc. (%)	Mean Rounds	Cumulative Uplink	Mean Drift Var ($\bar{\nu}$)
QNLI	$C = 20$ ($R_{\max} = 50$)	RoLoRA ($E = 10$)	81.10 ± 3.05	83.50 ± 1.18	33.0 ± 1.4	154.7 MB	2.212
		RoLoRA ($E = 20$)	83.32 ± 0.00	83.32 ± 0.00	$\mathbf{17.0 \pm 0.0}$	**79.7** MB	3.781
		FDA-RoLoRA ($e_{\max} = 20, k = 0.5$)	74.03 ± 9.88	82.64 ± 1.86	34.0 ± 4.0	159.4 MB	**1.848**
		FDA-RoLoRA ($e_{\max} = 20, k = 1.0$)	81.88 ± 2.05	$\mathbf{84.22 \pm 0.29}$	26.5 ± 5.5	124.2 MB	3.533
		FDA-RoLoRA ($e_{\max} = 40, k = 0.5$)	$\mathbf{83.87 \pm 0.00}$	84.15 ± 0.00	30.0 ± 0.0	140.6 MB	4.818
	$C = 50$ ($R_{\max} = 30$)	RoLoRA ($E = 10$)	$\mathbf{58.45 \pm 8.99}$	74.61 ± 7.18	30.0 ± 0.0	351.6 MB	4.120
		FDA-RoLoRA ($e_{\max} = 20, k = 0.5$)	50.13 ± 0.67	$\mathbf{75.69 \pm 1.17}$	30.0 ± 0.0	351.6 MB	**2.950**
MNLI	$C = 20$ ($R_{\max} = 50$)	RoLoRA ($E = 10$)	$\mathbf{73.61 \pm 0.06}$	73.74 ± 0.19	50.0 ± 0.0	234.4 MB	3.064
		RoLoRA ($E = 20$)	63.67 ± 0.00	72.56 ± 0.00	$\mathbf{20.0 \pm 0.0}$	**93.8** MB	5.949
		FDA-RoLoRA ($e_{\max} = 20, k = 0.5$)	71.80 ± 0.37	72.65 ± 1.01	41.0 ± 5.0	192.2 MB	**2.997**
		FDA-RoLoRA ($e_{\max} = 40, k = 0.5$)	73.43 ± 0.00	$\mathbf{74.21 \pm 0.00}$	35.0 ± 0.0	164.1 MB	6.448
	$C = 50$ ($R_{\max} = 30$)	RoLoRA ($E = 10$)	$\mathbf{55.38 \pm 5.21}$	55.52 ± 5.07	30.0 ± 0.0	351.6 MB	4.880
		FDA-RoLoRA ($e_{\max} = 20, k = 0.5$)	54.20 ± 4.50	$\mathbf{56.40 \pm 4.80}$	30.0 ± 0.0	351.6 MB	**3.750**

Table 5.3: Empirical evaluation on the statistical heterogeneity benchmark suite (Non-IID Dirichlet $\alpha = 0.5$) across GLUE tasks on `roberta-base`.

A rigorous inspection of Table 5.3 reveals several key findings regarding federated learning under statistical heterogeneity:

1. **Subspace Client Drift Variance Inflation**: Direct measurement of inter-client drift variance ($\bar{\nu}$) confirms that Dirichlet label skew ($\alpha = 0.5$) causes acute geometric dispersion on active coordinate tensors. On QNLI ($C = 20$), mean subspace drift variance expands from 0.503 in the homogeneous IID setting to 1.848 under Non-IID skew for FDA-RoLoRA ($e_{\max} = 20, k = 0.5$)—representing an immediate **3.67× surge** in parameter variance. Under aggressive local training ($k = 1.0$ or $E = 20$), variance surges even further to 3.533–4.818.

2. **Vulnerability of Rigid Schedules to Client Drift**: Under static local epoch schedules ($E = 10$ or $E = 20$), clients are forced to complete a predetermined number of local epochs regardless of whether their updates are rapidly diverging from the global model. In high-dimensional coordinate space, this leads to severe client over-specialization. On MNLI ($C = 20$), RoLoRA with $E = 20$ experiences severe post-peak degradation, dropping from 72.56% peak accuracy down to 63.67% final accuracy. When scaling to $C = 50$ clients under Dirichlet skew, both RoLoRA and FDA suffer from severe client dilution, where fixed schedules collapse to near majority-class baselines (55.38% on MNLI and 58.45% on QNLI).

3. **Adaptive Variance-Governed Halting**: In contrast to static schedules, FDA-RoLoRA monitors subspace drift variance at each epoch boundary. When severe label skew accelerates drift variance, the threshold condition halts local training dynamically within $[e_{\min}, e_{\max}]$, forcing immediate server synchronization. This adaptive braking mechanism limits the physical distance clients drift away from the consensus trajectory, suppressing mean drift variance (e.g., maintaining $\bar{\nu} = 1.848$ on QNLI vs. 3.781 for RoLoRA $E = 20$).

4. **Extended Communication Horizons Under Heterogeneity**: A key empirical dynamic in Table 5.3 is that Non-IID optimization traverses significantly longer communication round horizons (averaging 30 to 41 rounds) to reach peak generalization (84.22% on QNLI and 74.21% on MNLI). Under severe label skew, conflicting local gradient directions introduce non-monotonic validation accuracy oscillations in early rounds. While homogeneous runs exhibit rapid monotonic convergence that triggers early stopping after 10–18 rounds, alternating coordinate optimization under statistical heterogeneity requires an extended communication horizon for factor matrices A and B to iteratively reconcile divergent client subspaces. The configured early stopping parameters for non-IID ($R_{\min} = 30$ rounds, $W = 5, P = 5$) prevent premature termination during these transient drift fluctuations, allowing the model sufficient communication horizon to achieve robust consensus.

5.3.3 Cross-Setting Synthesis: Communication and Geometric Stability

Synthesizing the empirical findings across both the homogeneous (IID) and heterogeneous (Non-IID) benchmark suites underscores the dual architectural advantages of the FDA-RoLoRA framework:

1. **Spatial Communication Compression**: By decoupling the low-rank factors through alternating coordinate optimization, FDA-RoLoRA transmits only a single factor matrix (A or B) per communication round. This strictly halves the per-round uplink communication payload (4.69 MB per client for rank $r = 4$) compared to standard simultaneous LoRA (9.38 MB), saving hundreds of megabytes of aggregate bandwidth over extended training horizons.

2. **Strict Aggregation Linearity**: Because only one factor is active while the other is frozen, parameter aggregation at the server is mathematically linear, completely eliminating the bilinear aggregation error ($\frac{1}{N}\sum A_i B_i \neq \bar{A}\bar{B}$) that plagues standard FedLoRA.

3. **Temporal Adaptation to Heterogeneity**: In homogeneous settings, FDA-RoLoRA safely permits deeper local training, attaining optimal accuracy while accelerating round convergence. Under severe label skew, variance-driven dynamic synchronization throttles local epochs to maintain bounded drift, delivering robust foundation model fine-tuning across diverse distributed environments.

Chapter 6

Conclusion and Future Work

This thesis investigated the integration of Federated Dynamic Averaging (FDA) with Parameter-Efficient Low-Rank Adaptation (LoRA) in federated learning. By formulating and analyzing FDA-RoLoRA—which couples alternating low-rank parameter optimization with an internal dual-track threshold mechanism—this work examined how variance-driven dynamic synchronization can be coupled with alternating low-rank parameter optimization to jointly address spatial parameter overhead and rigid temporal synchronization schedules under statistical data heterogeneity.

6.1 Synthesis of Results and Methodological Takeaways

The theoretical formulation, algorithmic design, and empirical evaluation of FDA-RoLoRA yield several core methodological takeaways:

1. **Preservation of Aggregation Linearity**: Dynamic averaging cannot be naively coupled with standard federated LoRA architectures. In simultaneous factor adaptation, bilinear aggregation error—where $\frac{1}{N}\sum A_i B_i \neq (\frac{1}{N}\sum A_i)(\frac{1}{N}\sum B_i)$—distorts local client drift trajectories, causing variance monitoring to misdiagnose true model divergence. By adopting alternating coordinate optimization, the parameter space is restricted to a single factor (A or B) per communication round. This guarantees strict linearity in parameter updates, providing the necessary mathematical substrate for stable, decoupled subspace variance tracking without cross-matrix geometric distortion.

2. **Decoupled Subspace Geometric Monitoring**: Evaluating drift variance directly on isolated low-rank coordinate tensors eliminates the memory and communication overhead of full-model gradient approximations and high-dimensional sketching. By operating an internal dual-track threshold mechanism (θ_A, θ_B), the system regulates synchronization intervals via a responsive, parameter-free midpoint scaling controller that dynamically calibrates local epoch counts according to realized drift variance on the active coordinate subspace. Establishing the direct Standard update rule as the primary operational controller preserves algorithmic parsimony while maintaining responsive adaptation to local drift.

3. **Empirical Foundation Model Benchmarks**: Empirical evaluations across transformer foundation models (`roberta-base` on GLUE tasks) demonstrate that dy-

namic synchronization achieves substantial communication round savings while preserving model accuracy under statistical data heterogeneity.

6.2 Limitations

The methodology and conclusions presented in this work should be considered alongside several operational and experimental limitations:

1. **Empirical Scope and Model Diversity**: While evaluated on `roberta-base`, extensive multi-seed verification across diverse autoregressive language models and varying non-IID Dirichlet concentration regimes remains valuable future work.

2. **Simulated Distributed Environment**: Due to the lack of access to the original codebase from the RoLoRA literature, our framework relies on an independent, custom reimplementation. Furthermore, evaluations are conducted within a simulated multi-process federated environment on centralized GPU infrastructure (2× NVIDIA A10 GPUs), leaving real-world distributed network constraints—such as physical transit latency, asymmetric bandwidth, and stochastic client dropouts—unmodeled.

3. **Rank Scalability and Alternation Heuristics**: While exact Euclidean variance computation on compact coordinate tensors is computationally negligible for standard low ranks ($r = 8$), scaling to substantially higher ranks or wider layer coverage may eventually require approximate sketching techniques (such as the AMS sketch). Additionally, the alternating optimization protocol relies on a rigid deterministic alternation schedule, leaving adaptive or variance-driven matrix toggling criteria unaddressed.

6.3 Future Work

Building upon the findings established in this thesis, subsequent research should prioritize several promising directions:

1. **Broader Foundation Model Exploration**: Extending empirical benchmarking to larger autoregressive language models (such as LLaMA and Mistral families) and generative tasks across distributed GPU clusters will further quantify scalability under massive parameter regimes.

2. **Generalization Across Federated LoRA Variants**: Extending dynamic variance monitoring to alternative parameter-efficient federated paradigms. Key directions include investigating **FDA + SLoRA** [3] for coordinate-sparse parameter exchanges, **FDA + FFA-LoRA** [36] with permanently frozen down-projections, and examining whether geometric variance tracking can help diagnose and mitigate client drift in standard simultaneous **FDA + FedLoRA**. Furthermore, developing adaptive rank allocation algorithms, where the rank r dynamically expands or contracts in response to observed subspace variance, represents a natural extension.

3. **Advanced Controller Design**: While this thesis adopts the direct, parameter-free Standard update rule, subsequent studies should systematically examine whether advanced closed-loop feedback controllers (such as PID-regulated threshold scaling and variance-acceleration damping) can provide enhanced stability under severe distribution shifts or deeper foundation models. Concurrently, developing formal convergence proofs for alternating dynamic averaging under non-convex loss topologies constitutes an important theoretical milestone.

4. **Optimization and System Hardening**: From an optimization perspective, exploring asymmetric learning rate schedules tailored independently for down-projection (A) and up-projection (B) matrices offers potential convergence gains. In addition, architectural integration with low-bit quantization frameworks (such as QLoRA [9]) and deployment over physical edge testbeds will further establish operational feasibility. Finally, incorporating FDA-RoLoRA into established open-source federated benchmarking platforms, such as **FederatedScope-LLM** [21], will facilitate broader community adoption and standardized evaluation across emerging foundation models.

In conclusion, this work demonstrates that parameter-efficient federated fine-tuning can effectively transcend rigid synchronization schedules through variance-driven dynamic averaging. By establishing that alternating coordinate optimization provides a mathematically consistent substrate for geometric variance monitoring, this thesis provides a principled and communication-efficient framework for federated adaptation.

Bibliography

[1] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

[2] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC)*, pages 20–29, 1996.

[3] Sara Babakniya, Souvik Ahmed, Yelong Shen, Bolin Ding, and Dawn Song. SLoRA: Federated Parameter-Efficient Fine-Tuning with Sparse Low-Rank Adaptation. *arXiv preprint arXiv:2311.12310*, 2023.

[4] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258*, 2021.

[5] Léon Bottou. Large-Scale Machine Learning with Stochastic Gradient Descent. In *Proceedings of COMPSTAT'2010*, 2010.

[6] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, 2020.

[7] Shuangyi Chen, Yuanxin Guo, Yue Ju, Hardik Dalal, Zhongwen Zhu, and Ashish Khisti. Robust Federated Finetuning of LLMs via Alternating Optimization of LoRA. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.

[8] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Andrew Senior, Paul Tucker, Ke Yang, Quoc V Le, et al. Large Scale Distributed Deep Networks. In *Advances in Neural Information Processing Systems*, 2012.

[9] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized Large Language Models. In *Advances in Neural Information Processing Systems*, 2023.

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019.

[11] Georgios Frangias. Federated learning at TensorFlow using the geometric approach. Diploma thesis, Technical University of Crete, 2023.

[12] Georgios Frangias. Federated learning at TensorFlow using the geometric approach. Diploma thesis, Technical University of Crete, 2023.

[13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT press, 2016.

[14] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *International Conference on Machine Learning*, 2019.

[15] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification. *arXiv preprint arXiv:1909.06335*, 2019.

[16] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022.

[17] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and Open Problems in Federated Learning. *Foundations and Trends in Machine Learning*, 14(1–2):1–210, 2021.

[18] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. In *International Conference on Machine Learning*, 2020.

[19] Kanakis Kelaidis. Federated LoRA-tuning for large language models. Diploma thesis, Technical University of Crete, 2025.

[20] Diederik P Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015.

[21] Weirui Kuang, Bingyi Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. FederatedScope-LLM: A Comprehensive Package for Federated Fine-Tuning of Large Language Models. *arXiv preprint arXiv:2309.00363*, 2023.

[22] Fan Lai, Xiangfeng Zhu, Harsha V Madhyastha, and Mosharaf Chowdhury. Oort: Efficient Federated Learning via Guided Participant Selection. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 269–282, 2021.

[23] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated Optimization in Heterogeneous Networks. In *Proceedings of Machine Learning and Systems*, 2020.

[24] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. On the Convergence of FedAvg on Non-IID Data. In *International Conference on Learning Representations*, 2020.

[25] Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

[26] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*, 2019.

[27] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019.

[28] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Artificial Intelligence and Statistics*, pages 1273–1282. PMLR, 2017.

[29] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

[30] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečnỳ, Sanjiv Kumar, and H Brendan McMahan. Adaptive Federated Optimization. In *International Conference on Learning Representations*, 2021.

[31] Herbert Robbins and Sutton Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, pages 400–407, 1951.

[32] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[33] Izchak Sharfman, Assaf Schuster, and Daniel Keren. A geometric approach to monitoring threshold functions over distributed data streams. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, pages 301–312, 2006.

[34] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1631–1642, 2013.

[35] Omer Subasi, Oceane Bel, Joseph Manzano, and Kevin Barker. The landscape of modern machine learning: A review of machine, distributed and federated learning. *arXiv preprint arXiv:2312.03120*, 2023.

[36] Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. Improving LoRA in Privacy-preserving Federated Learning. In *The Twelfth International Conference on Learning Representations*, 2024.

[37] Michail Theologitis. Algorithms for online federated machine learning. Diploma thesis, Technical University of Crete, 2023.

[38] Michail Theologitis, Vasilis Samoladas, and Antonios Deligiannakis. FDA-Opt: Communication-Efficient Federated Fine-Tuning of Language Models. *arXiv preprint arXiv:2505.04535*, 2025. Code available at `https://github.com/michaeltheologitis/FDA-Opt`.

[39] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*, 2023.

[40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017.

[41] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *International Conference on Learning Representations*, 2019.

[42] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H Vincent Poor. Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization. In *Advances in Neural Information Processing Systems*, volume 33, pages 7611–7623, 2020.

[43] Ziyao Wang, Zheyu Shen, Yexiao He, Guoheng Sun, Hongyi Wang, Lingjuan Lyu, and Ang Li. FLoRA: Federated Fine-Tuning Large Language Models with Heterogeneous Low-Rank Adaptations. In *NeurIPS*, 2024.

[44] Adina Williams, Nikita Nangia, and Samuel R Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 1112–1122, 2018.

[45] Haibo Yang et al. Federated Low-Rank Adaptation: A Comprehensive Survey. *arXiv preprint arXiv:2404.06423*, 2025.

[46] Mikhail Yurochkin, Mayank Agarwal, Soumya Banerjee, Rebecca Chen, Dennis Sun, and Yasaman Khazaeni. Bayesian Nonparametric Federated Learning of Neural Networks. In *International Conference on Machine Learning (ICML)*, pages 7252–7261, 2019.

[47] Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. Towards Building the Federated GPT: Federated Instruction Tuning. *arXiv preprint arXiv:2305.05644*, 2023.

[48] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. In *International Conference on Learning Representations*, 2023.

[49] Yang Zhao et al. LoRA-the-Explorer: An Efficient Framework for Training and Merging Low-Rank Adapters in Federated Learning. *arXiv preprint*, 2024.

[50] Zhuangdi Zhu, Junlin Hong, and Shiyu Ji. Federated Learning on Non-IID Data: A Survey. *Neurocomputing*, 465:371–390, 2021.